MACHINE LEARNING LAB

ETCS - 454

Faculty Name: Mr Anupam Kumar

Name: Aman Chauhan

Roll No: 14614802719

Semester: 8th

Group: 8C7

Maharaja Agrasen Institute of Technology, PSP area,
Sector - 22, Rohini, New Delhi - 110085

(Affiliated to Guru Gobind Singh Indraprastha
University, New Delhi)

Introduction to Machine Learning Lab

Course Objective:

The main objective of this course is to ensure that students understand the working of different machine learning techniques in practice for popular algorithms like c4.5 algorithm, Naïve Bayes, decision tree, random forest or neural network get applied on some statistical data**.**

Course Outcome:

At the end of the course, a student will be able to:

C454.1: Study and Implement the Supervised learners (Naive Bayes) and Unsupervised Learners

C454.2: Estimate the precision, recall, F-measure, and accuracy of classifier methods on some dataset. (breast cancer) using 5-fold cross-validation.

C454.3: To understand datasets of various problems and evaluate their attributes for training, testing, and validations. Estimate overfitting if exists.

C454.4: Develop a machine learning method to Predict stock prices or education of infants etc, based on past price variation, to predict how people would rate movies, books, etc**C454.5:** Select two Programming languages to implement the same problem under different classifier algorithm. Use different functions to implement the concept.

C454.6: Evaluate performance by measuring the sum of different distance algorithms of each example from its class center test performance as a function of the number of clusters.

Department of Computer Science and Engineering

Rubrics for Lab Assessment

Rubrics		0	1	2	3
		Missing	**Inadequate**	**Needs Improvement**	**Adequate**
R1	Is able to identify the problem to be solved and define the objectives of the experiment.	No mention is made of the problem to be solved.	An attempt is made to identify the problem to be solved but it is described in a confusing manner, objectives are not relevant, objectives contain technical/ conceptual errors or objectives are not measurable.	The problem to be solved is described but there are minor omissions or vague details. Objectives are conceptually correct and measurable but may be incomplete in scope or have linguistic errors.	The problem to be solved is clearly stated. Objectives are complete, specific, concise, and measurable. They are written using correct technical terminology and are free from linguistic errors.
R2	Is able to design a reliable experiment that solves the problem.	The experiment does not solve the problem.	The experiment attempts to solve the problem but due to the nature of the design the data will not lead to a reliable solution.	The experiment attempts to solve the problem but due to the nature of the design there is a moderate chance the data will not lead to a reliable solution.	The experiment solves the problem and has a high likelihood of producing data that will lead to a reliable solution.
R3	Is able to communicate the details of an experimental procedure clearly and completely.	Diagrams are missing and/or experimental procedure is missing or extremely vague.	Diagrams are present but unclear and/or experimental procedure is present but important details are missing.	Diagrams and/or experimental procedure are present but with minor omissions or vague details.	Diagrams and/or experimental procedure are clear and complete.
R4	Is able to record and represent data in a meaningful way.	Data are either absent or incomprehensible.	Some important data are absent or incomprehensible.	All important data are present, but recorded in a way that requires some effort to comprehend.	All important data are present, organized and recorded clearly.
R5	Is able to make a judgment about the results of the experiment.	No discussion is presented about the results of the experiment .	A judgment is made about the results, but it is not reasonable or coherent.	An acceptable judgment is made about the result, but the reasoning is flawed or incomplete.	An acceptable judgment is made about the result, with clear reasoning. The effects of assumptions and experimental uncertainties are considered.

MACHINE LEARNING LAB PRACTICAL RECORD

PAPER CODE: **ETCS – 454**

Name: **AMAN CHAUHAN**

University Roll No.: **14614802719**

Group: **8C7**

Branch: **CSE**

S.No	EXPERIMENT NAME	DATE	MARKS					Total Marks	Signature
			R1	R2	R3	R4	R5		
1.	Introduction to Machine learning lab with tools (hands-on Weka)								
2.	Understanding of Machine learning algorithms List of Databases								
3.	Implement K means Algorithm using WEKA Tool. Implement the K-means algorithm and apply it to the data you selected. Evaluate performance by measuring the sum of Euclidean distance of each example from its class Center. Test the performance of the algorithm as a function of the parameter k.								
4.	Study of Databases and understanding attributes evaluation in regard to problem description								
5.	Working of Major Classifiers, a) Naïve Bayes b) Decision Tree c)CART d) ARIMA (using (e) linear and logistics regression (f) Support vector machine (g) KNN (datasets can be: Breast Cancer data file or Reuters data set).								
6.	Design a prediction model for Analysis of round trip Time of Flight measurement from a supermarket using the random forest, Naïve Bayes, etc.								
7.	Implement supervised learning (KNN classification) Estimate the accuracy of using 5-fold cross-validation. Choose the appropriate options for missing values.								

8.	Introduction to R. Be aware of the basics of machine learning methods in R.								
9.	Build and develop a model in R for a particular classifier (random Forest).								
10.	Develop a machine learning method using Neural Networks in python to Predict stock prices based on past price variation,								
11.	Case Study of RMS Titanic Database to predict survival on the basis of the decision tree, Logistic Regression, KNN or k-Nearest Neighbors, Support Vector Machines								
12.	Understanding of Indian education in Rural villages to predict whether a girl child will be sent to school or not?.								
13.	Understanding of dataset of contact patterns among students collected in the National University of Singapore.								

EXPERIMENT – 1

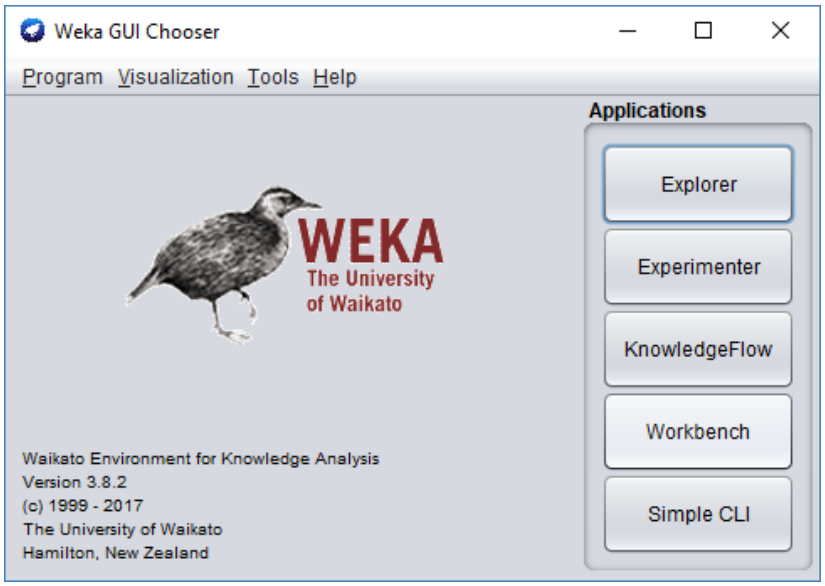
AIM - Introduction to machine learning lab with tools (hands on Weka).

THEORY-

Weka is a data mining/machine learning application and is being developed by Waikato University in New Zealand. The purpose of this article is to teach you how to use the Weka Explorer, classify a dataset with Weka, and visualize the results.

(a) Start Weka

Start Weka. This may involve finding it in program launcher or double clicking on the weka.jar file. This will start the Weka GUI Chooser. The Weka GUI Chooser lets you choose one of the Explorer, Experimenter, Knowledge Explorer and the Simple CLI (command line interface).



Weka GUI Chooser

Click the "***Explorer***" button to launch the Weka Explorer. This GUI lets you load datasets and run classification algorithms. It also provides other features, like data filtering, clustering, association rule extraction, and visualization, but we won't be using these features right now.

(b). Open the data/ Dataset(sample iris.arff)

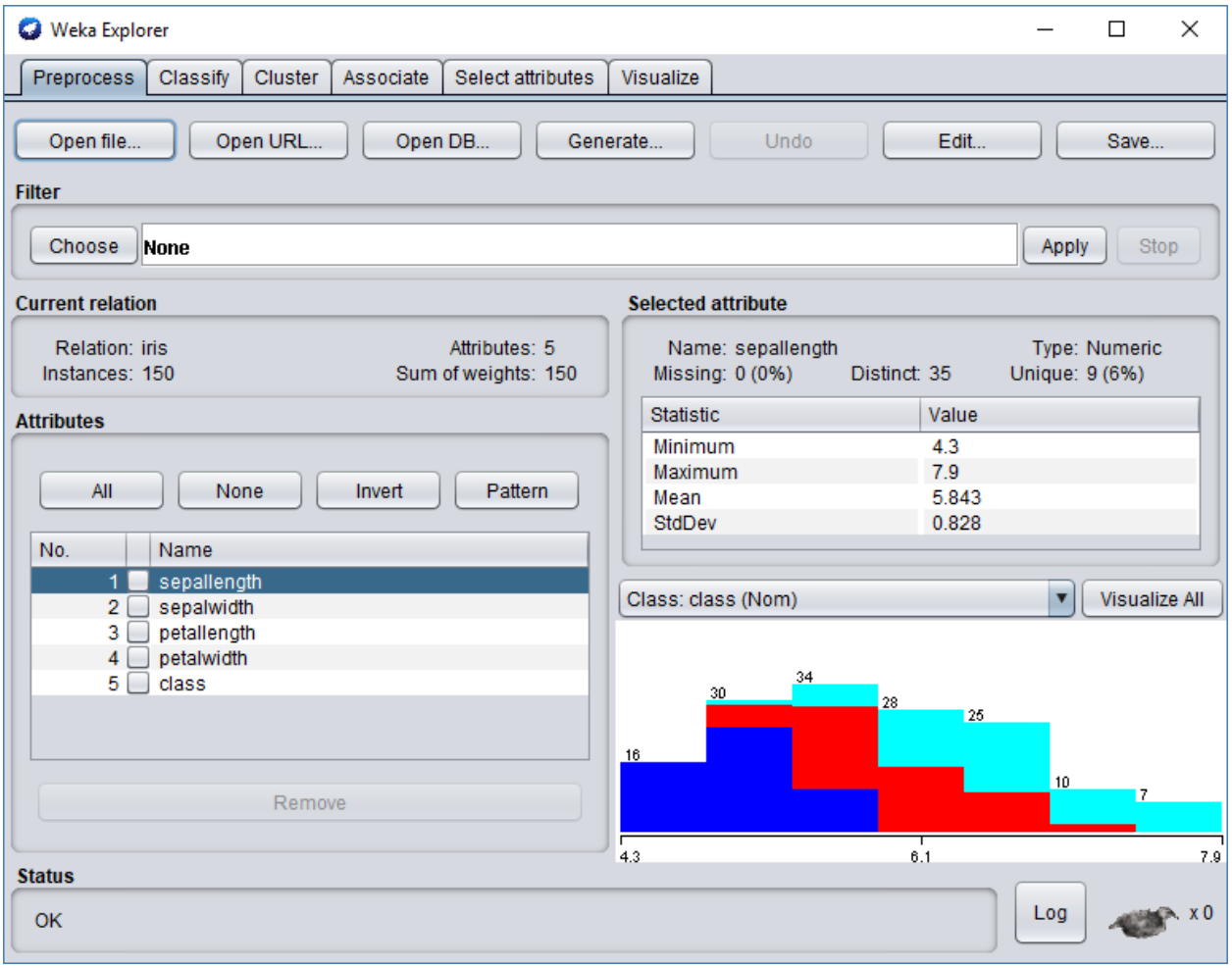
Dataset

A comma-separated list. A set of data items, the dataset, is a very basic concept of machine learning. A dataset is roughly equivalent to a two-dimensional spreadsheet or database table. In WEKA,it is implemented by the weka.core.Instances class. A dataset is a collection of examples, each one of class weka.core.Instance. Each Instance consists of a number of attributes, any of which can be nominal (= one of a predefined list of values), numeric (= a real or integer number)or a string

(= an arbitrary long list of characters, enclosed in ”double quotes”). Additional types are date and relational, which are not covered here but in the ARFF chapter. The external representation of an Instances class is an ARFF file, which consists of a header describing the attribute types and the data.

Click the “***Open file…***” button to open a data set and double click on the “***data***” directory.

Weka provides a number of small common machine learning datasets that you can use to practice on. Select the “***iris.arff***” file to load the Iris dataset.



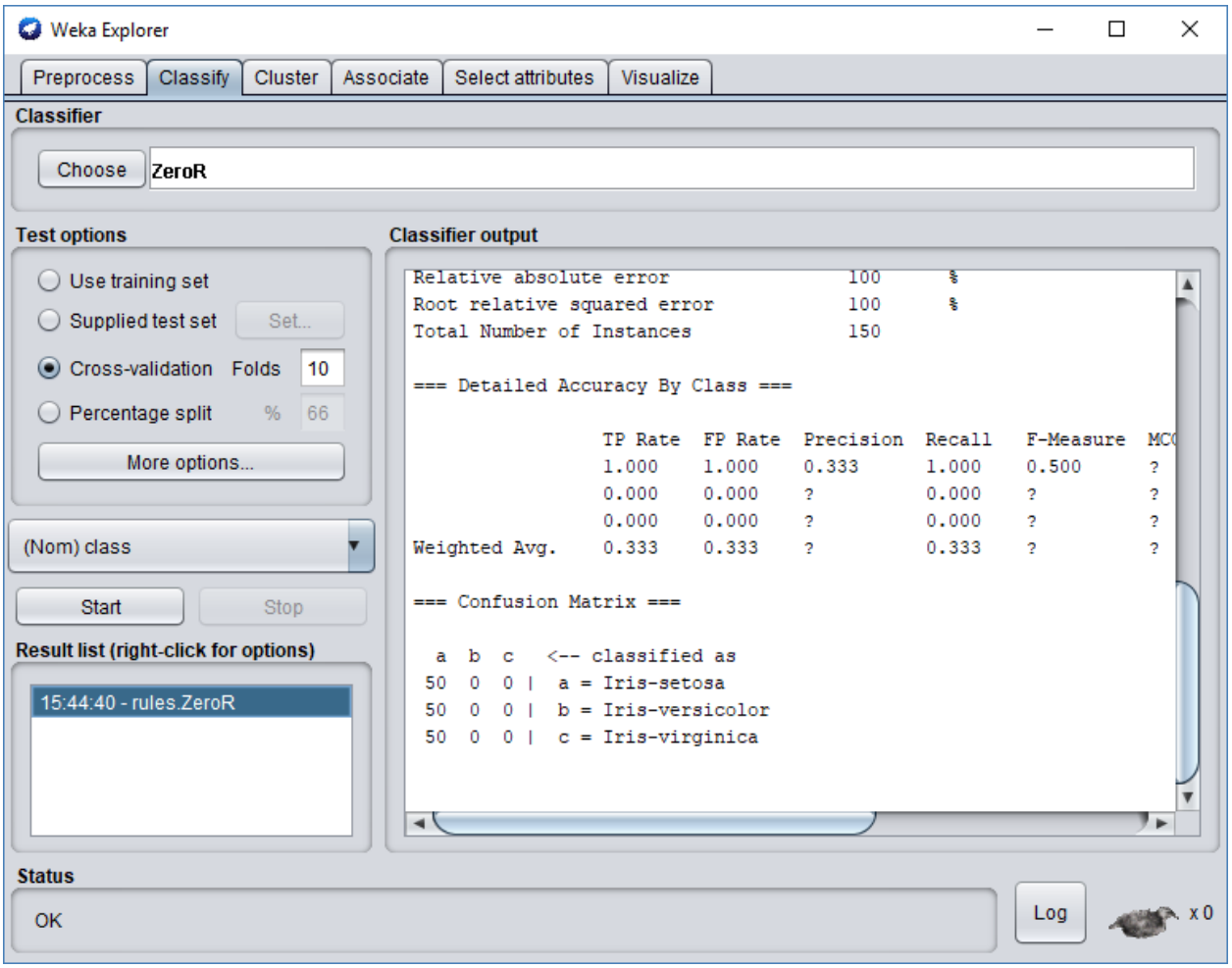
Weka Explorer Interface with the Iris dataset loaded .The Iris Flower dataset is a famous dataset from statistics and is heavily borrowed by researchers in machine learning. It contains 150 instances (rows) and 4 attributes (columns) and a class attribute for the species of iris flower (one of setosa, versicolor, and virginica).In our example, we have not mentioned the attribute type string, which defines ”double quoted” string attributes for text mining. In recent WEKA versions, date/time attribute types are also supported. By default, the last attribute is considered the class/target variable, i.e. the attribute which should be predicted as a function of all other attributes. If this is not the case, specify the target variable via -c. The attribute numbers are one- based indices, i.e. -c 1 specifies the first attribute. Some basic statistics and validation of given ARFF files can be obtained via the main() routine of weka.core.Instances:

Classifier

Any learning algorithm in WEKA is derived from the abstract weka.classifiers.Classifier class. Surprisingly little is needed for a basic classifier: a routine Which gen- erates a classifier modelfrom a training dataset (= buildClassifier) and another routine which evaluates the generated model on an unseen test dataset (= classifyInstance), or generates a probability distribution for all classes (= distributionForInstance).A classifier model is an arbitrary complex mapping from all-but-one dataset attributes to the class attribute.

(c) Select and Run an Algorithm

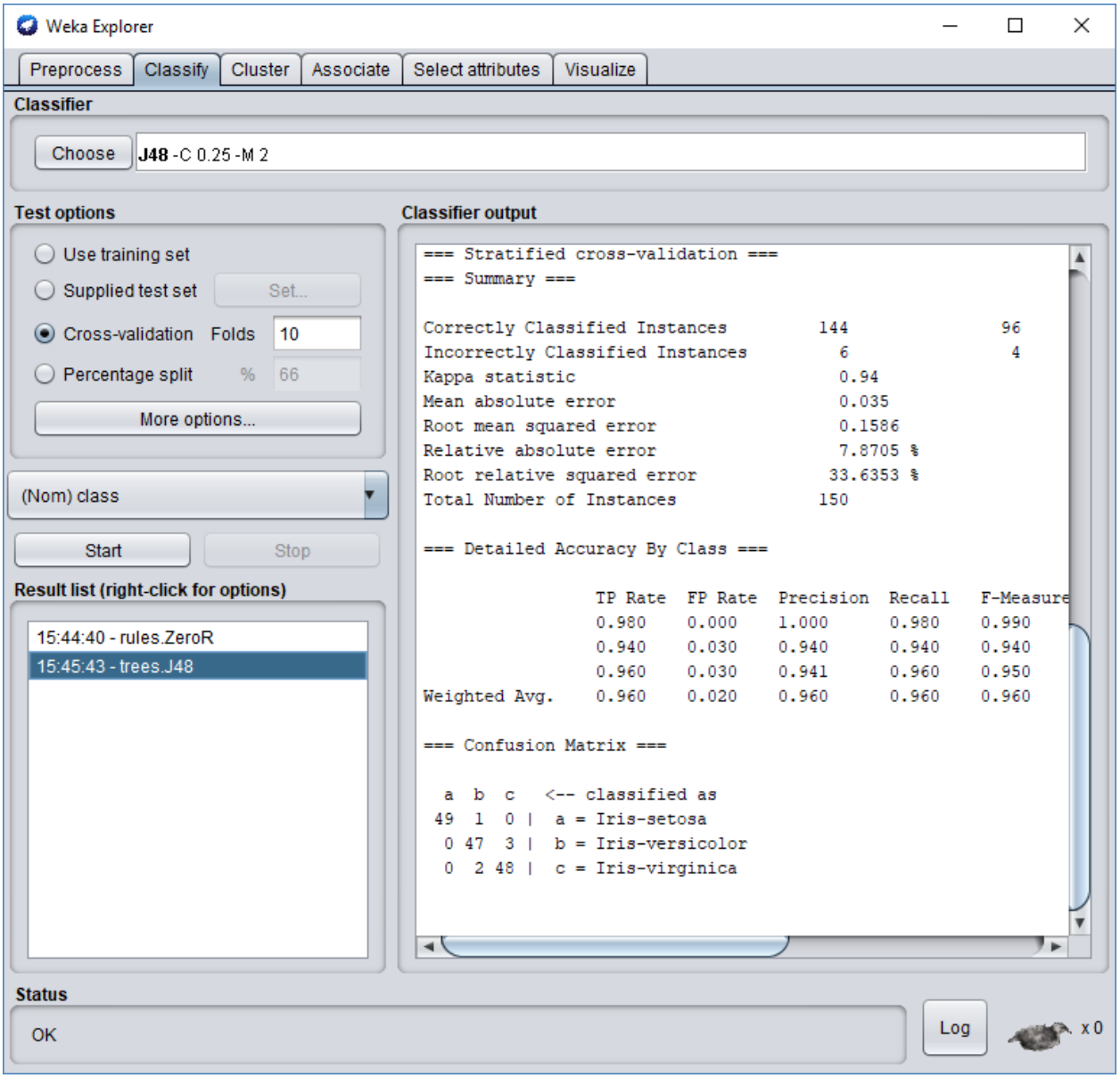
Now that you have loaded a dataset, it's time to choose a machine learning algorithm to model the problem and make predictions. Click the "***Classify***" tab. This is the area for running algorithms against a loaded dataset in Weka. You will note that the "***ZeroR***" algorithm is selectedby default. Click the "***Start***" button to run this algorithm.



Weka Results for the ZeroR algorithm on the Iris flower dataset. The ZeroR algorithm selects the majority class in the dataset (all three species of iris are equally present in the data, so it picks the first one: setosa) and uses that to make all predictions. This is the baseline for the dataset and the

measure by which all algorithms can be compared. The result is 33%, as expected (3 classes, each equally represented, assigning one of the three to each prediction results in 33% classification accuracy).

You will also note that the test options selects Cross Validation by default with 10 folds. This means that the dataset is split into 10 parts: the first 9 are used to train the algorithm, and the 10this used to assess the algorithm. This process is repeated, allowing each of the 10 parts of the split dataset a chance to be the held-out test set. Click the "Choose" button in the "Classifier" section and click on "trees" and click on the "J48" algorithm. This is an implementation of the C4.8 algorithm in Java ("J" for Java, 48 for C4.8, hence the J48 name) and is a minor extension to the famous C4.5 algorithm. Click the "***Start***" button to run the algorithm.



Weka J48 algorithm results on the Iris flower dataset

(c) **Review Results**

After running the J48 algorithm, you can note the results in the "Classifier output" section.

The algorithm was run with 10-fold cross-validation: this means it was given an opportunity to make a prediction for each instance of the dataset (with different training folds) and the presented result is a summary of those predictions.

```
=== Summary ===

Correctly Classified Instances         144               96      %
Incorrectly Classified Instances         6                4      %
Kappa statistic                          0.94
Mean absolute error                      0.035
Root mean squared error                  0.1586
Relative absolute error                  7.8705 %
Root relative squared error             33.6353 %
Total Number of Instances              150

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.980    0.000    1.000      0.980    0.990      0.985    0.990     0.987     Iris-setosa
                 0.940    0.030    0.940      0.940    0.940      0.910    0.952     0.880     Iris-versicolor
                 0.960    0.030    0.941      0.960    0.950      0.925    0.961     0.905     Iris-virginica
Weighted Avg.    0.960    0.020    0.960      0.960    0.960      0.940    0.968     0.924

=== Confusion Matrix ===

  a  b  c   <-- classified as
 49  1  0 |  a = Iris-setosa
  0 47  3 |  b = Iris-versicolor
  0  2 48 |  c = Iris-virginica
```

Just the results of the J48 algorithm on the Iris flower dataset in Weka.

Firstly, note the Classification Accuracy. You can see that the model achieved a result of 144/150 correct or 96%, which seems a lot better than the baseline of 33%.Secondly, look at the Confusion Matrix. You can see a table of actual classes compared to predicted classes and you can see that there was 1 error where an Iris-setosa was classified as an Iris-versicolor, 2 cases where Iris-virginica was classified as an Iris-versicolor, and 3 cases where an Iris-versicolor was classified as an Iris-setosa (a total of 6 errors). This table can help to explain the accuracy achieved by the algorithm.

VIVA VOCE QUESTIONS

Q1. What is a logistic function? What is the range of values of a logisticfunction?
Ans. $f(z) = 1/(1+e^{-z})$
The values of a logistic function will range from 0 to 1. The values of Z will vary from -infinity to +infinity.

Q2. Why is logistic regression important?
Ans. Logistic regression is important because it can convert the values of logits (logodds), which can range from -infinity to +infinity to a range between 0 and 1. As logistic functions output the probability of occurrence of an event, it can be applied to many real-life scenarios. It is for this reason that the logistic regression model is very popular.

Q3. What is the formula for the logistic regression function?
Ans. $f(z) = 1/(1+e^{-(\alpha+1X1+2X2+....+kXk)})$

Q4. Why can't linear regression be used in place of logistic regression for binary classification?

Ans. The reasons why linear regressions cannot be used in case of binary classification are as follows:

Distribution of error terms: The distribution of data in case of linear and logistic regression is different. Linear regression assumes that error terms are normally distributed. In case of binary classification, this assumption does not hold true.

Model output: In linear regression, the output is continuous. In case of binary classification, an output of a continuous value does not make sense. For binary classification problems, linear regression may predict values that can go beyond 0 and 1. If we want the output in the form of probabilities, which can be mapped to two different classes, then its range should be restricted to 0 and 1. As the logistic regression model can output probabilities with logistic/sigmoid function, it is preferred over linear regression.

Variance of Residual errors: Linear regression assumes that the variance of random errors is constant. This assumption is also violated in case of logistic regression.

Q5. What are the assumptions of logistic regression?
Ans. The assumptions of logistic regression include:

a. The outcome variable should be binary or categorical.
b. The relationship between predictors and the log odds of the outcome should be linear.
c. There should be little or no multicollinearity among the predictor variables.
d. The observations should be independent of each other

EXPERIMENT – 2

AIM - Understanding of Machine learning algorithms.

Machine learning algorithms are programs that can learn from data and improve from experience, without human intervention. Learning tasks may include learning the function that maps the input to the output, learning the hidden structure in unlabeled data; or 'instance-based learning', where a class label is produced for a new instance by comparing the new instance (row) to instances from the training data, which were stored in memory.

Types of Machine Learning Algorithms

There are 3 types of machine learning (ML) algorithms:

1. Supervised Learning Algorithms:

Supervised learning uses labeled training data to learn the mapping function that turns input variables (X) into the output variable (Y). In other words, it solves for f in the following equation: $Y = f(X)$

This allows us to accurately generate outputs when given new inputs.

We'll talk about two types of supervised learning: classification and regression.

Classification is used to predict the outcome of a given sample when the output variable is in the form of categories. A classification model might look at the input data and try to predict labels like "sick" or "healthy."

Regression is used to predict the outcome of a given sample when the output variable is in the form of real values. For example, a regression model might process input data to predict the amount of rainfall, the height of a person, etc.

The first 5 algorithms that we cover in this blog – Linear Regression, Logistic Regression, CART, Naïve-Bayes, and K-Nearest Neighbors (KNN) — are examples of supervised learning.

Ensembling is another type of supervised learning. It means combining the predictions of multiple machine learning models that are individually weak to produce a more accurate prediction on a new sample. Algorithms 9 and 10 of this article — Bagging with Random Forests, Boosting with XGBoost — are examples of ensemble techniques.

2. Unsupervised Learning Algorithms:

Unsupervised learning models are used when we only have the input variables (X) and no corresponding output variables. They use unlabeled training data to model the underlying structure of the data.

We'll talk about three types of unsupervised learning:

Association is used to discover the probability of the co-occurrence of items in a collection. It is extensively used in market-basket analysis. For example, an association model might be used to discover that if a customer purchases bread, s/he is 80% likely to also purchase eggs.

Clustering is used to group samples such that objects within the same cluster are more similar to each other than to the objects from another cluster.

Dimensionality Reduction is used to reduce the number of variables of a data set while ensuring that important information is still conveyed. Dimensionality Reduction can be done using Feature Extraction methods and Feature Selection methods. Feature Selection selects a subset of the original variables. Feature Extraction performs data transformation from a high-dimensional space to a low-dimensional space. Example: PCA algorithm is a Feature Extraction approach.

Apriori, K-means, PCA — are examples of unsupervised learning.

3. Reinforcement learning:

Reinforcement learning is a type of machine learning algorithm that allows an agent to decide the best next action based on its current state by learning behaviors that will maximize a reward.

Reinforcement algorithms usually learn optimal actions through trial and error. Imagine, for example, a video game in which the player needs to move to certain places at certain times to earn points. A reinforcement algorithm playing that game would start by moving randomly but, over time through trial and error, it would learn where and when it needed to move the in-game character to maximize its point total.

VIVA VOCE QUESTION

Q. Why is accuracy not a good measure for classification problems?

Ans. Accuracy is not a good measure for classification problems because it gives equal importance to both false positives and false negatives. However, this may not be the case in most business problems. For example, in case of cancer prediction, declaring cancer as benign is more serious than wrongly informing the patient that he is suffering from cancer. Accuracy gives equal importance to both cases and cannot differentiate between them.

Q. What are false positives and false negatives?

Ans. False positives are those cases in which the negatives are wrongly predicted aspositives. For example, predicting that a customer will churn when, in fact, he is notchurning.

False negatives are those cases in which the positives are wrongly predicted as negatives.For example, predicting that a customer will not churn when, in fact, he churns.

Q. What are the true positive rate (TPR), true negative rate (TNR), false positive rate (FPR), and false negative rate (FNR)?

Ans. TPR refers to the ratio of positives correctly predicted from all the true labels. Insimple words, it is the frequency of correctly predicted true labels. TPR = TP/TP+FN

TNR refers to the ratio of negatives correctly predicted from all the false labels. It is the frequency of correctly predicted false labels. TNR = TN/TN+FP

FPR refers to the ratio of positives incorrectly predicted from all the true labels. It is the frequency of incorrectly predicted false labels. FPR = FP/TN+FP

FNR refers to the ratio of negatives incorrectly predicted from all the false labels. It is the frequency of incorrectly predicted true labels. FNR = FN/TP+FN

Q. What are precision and recall?

Ans. Precision is the proportion of true positives out of predicted positives. To put it in another way, it is the accuracy of the prediction. It is also known as the ‘positive predictive value’. Precision = TP/TP+FP

Q. What is the difference between supervised and unsupervised learning?

Ans. Supervised learning is a type of machine learning where the algorithm learns from labeled data, where the input data is accompanied by the corresponding output labels. The algorithm learns to predict the output labels for new, unseen data based on the patterns observed in the labeled training data. In contrast, unsupervised learning is a type of machine learning where the algorithm learns from unlabeled data. The algorithm explores the patterns and structures within the data without any specific output labels.

EXPERIMENT – 3

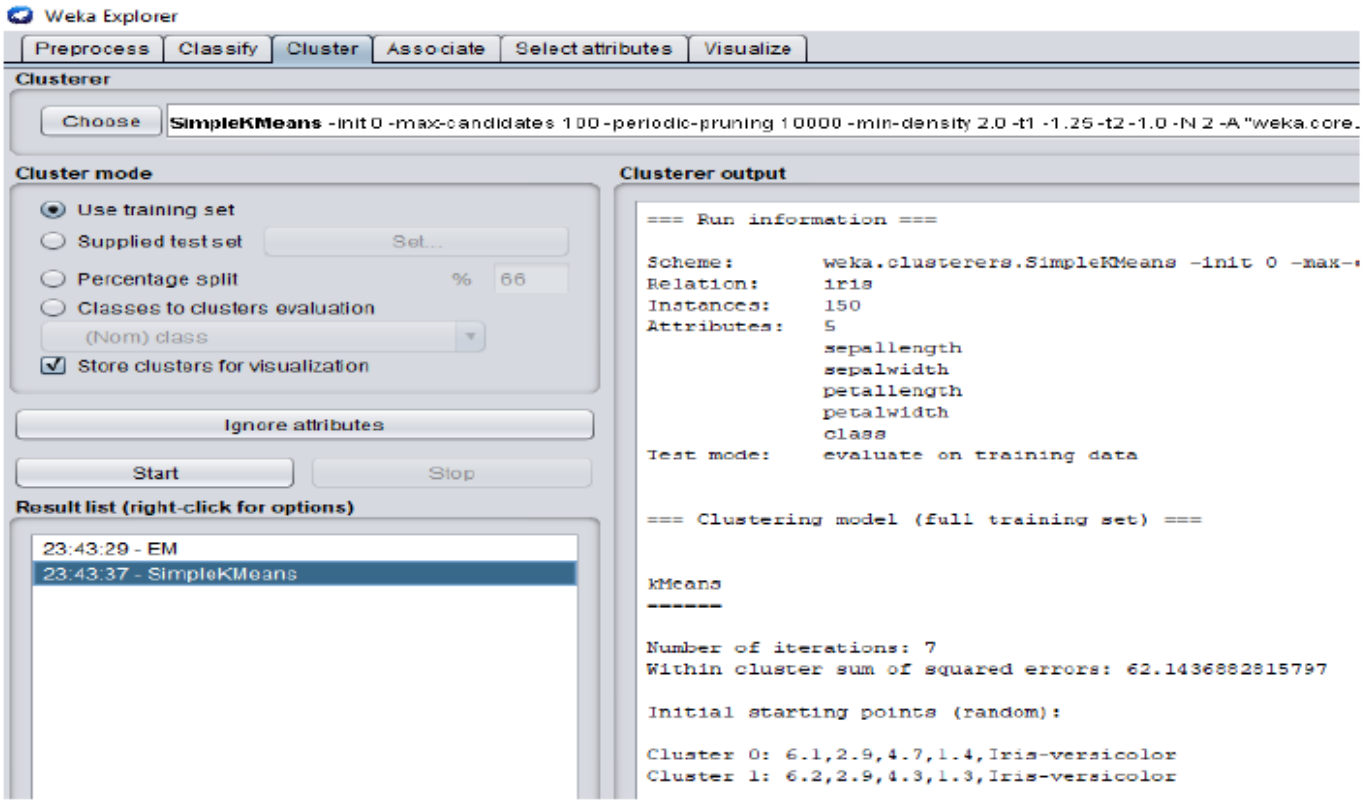
AIM - Implement K means Algorithm using WEKA Tool. Implement the K-means algorithmand apply it to the data you selected. Evaluate performance by measuring the sum of Euclidean distance of each example from its class center. Test the performance of the algorithm as a function of the parameter k.

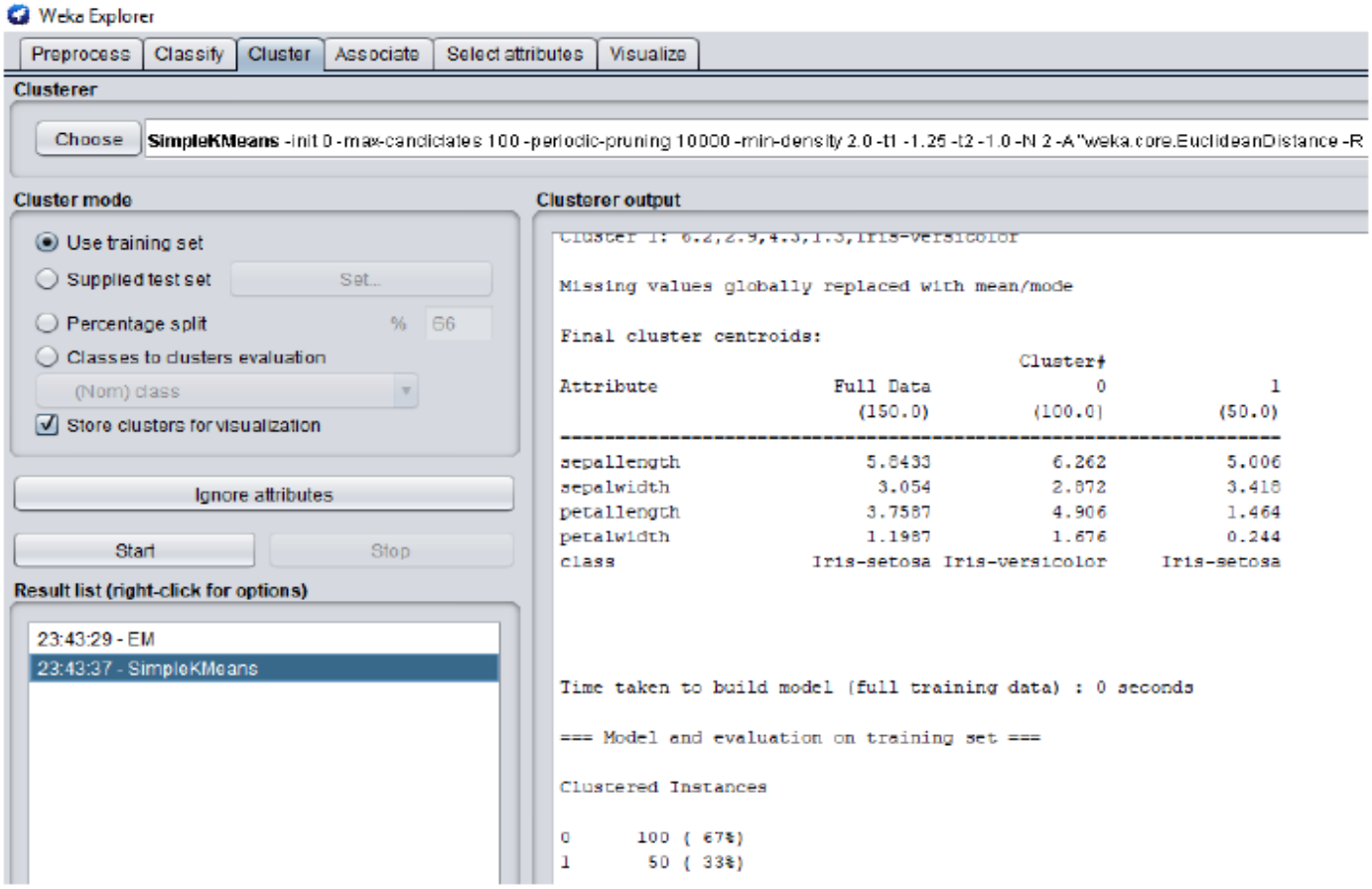
Theory:

K means algorithm is an iterative algorithm that tries to partition the dataset into K pre-defined distinct non-overlapping subgroups (clusters) where each data point belongs to only one group. It tries to make the inter-cluster data points as similar as possible while also keeping the clusters as different (far) as possible. It assigns data points to a cluster such that the sum of the squared distance between the data points and the cluster's centroid (arithmetic mean of all the data points that belong to that cluster) is at the minimum. The less variation we have within clusters, the more homogeneous (similar) the data points are within the same cluster.

The way k means algorithm works is as follows:

1. Specify number of clusters K.
2. Initialize centroids by first shuffling the dataset and then randomly selecting K data points for the centroids without replacement.
3. Keep iterating until there is no change to the centroids. i.e assignment of data points to clusters isn't changing.
   - Compute the sum of the squared distance between data points and all centroids.
   - Assign each data point to the closest cluster (centroid).
   - Compute the centroids for the clusters by taking the average of the all data points that belong to each cluster.





The above information shows the result of k-means clustering Methods using WEKA tool. After that we saved the result, the result will be saved in the ARFF file format. We also open this filein the ms excel.

VIVA VOICE QUESTIONS

Q. Why is "Naive" Bayes naive?

Ans. Despite its practical applications, especially in text mining, Naive Bayes is considered "Naive" because it makes an assumption that is virtually impossible to see in real-life data: the conditional probability is calculated as the pure product of the individual probabilities of components. This implies the absolute independence of features — a condition probably never met in real life.

Q. What is SVM?

Ans. A Support Vector Machine (SVM) is a discriminative classifier formally defined by a separating hyperplane. In other words, given labelled training data (supervised learning), the algorithm outputs an optimal hyperplane which categorizes new examples. In two dimensional space this hyperplane is a line dividing a plane in two parts where in each class lay in either side.

Q. What is perceptron?

Ans. In machine learning, the perceptron is an algorithm for supervised learning of binary classifiers. A binary classifier is a function which can decide whether or not an input, represented by a vector of numbers, belongs to some specific class.

Q. What are Exponential families?

Ans. Exponential families are a broad class of probability distributions which includesmany basic distributions such as Bernoullis and Gaussians, as well as Markov randomfields. What they have in common is that the distributions can be represented in termsof log-linear functions of sufficient statistics.

Q. How does the K-means algorithm initialize centroids?

Ans: The K-means algorithm initializes centroids by randomly selecting K data points from the dataset as the initial centroids. Alternatively, other initialization methods like k-means++ can be used to improve the efficiency and quality of the final clustering.

EXPERIMENT – 4

AIM: Study of Databases and understanding attributes evaluation in regard to problem description.

There are lot of data files that store attributes details of problem description and they store data ineither of formats

1. CSV- Comma Separated Value
2. arff
3. Excel -xls

There is list of following **arff** files shown below. These files are evaluated and analysed to get results on basis of data provided in files.

1) Airline
2) Breast-cancer
3) Contact-lenses
4) Cpu
5) Credit-g

1. Airline:

Monthly totals of international airline passengers (in thousands) for 1949-1960.

```
@relation airline_passengers
@attributepassenger_numbers numeric
@attribute Date date 'yyyy-MM-dd'

@data
112,1949-01-01
118,1949-02-01
132,1949-03-01
129,1949-04-01
121,1949-05-01
135,1949-06-01
148,1949-07-01
148,1949-08-01
432,1960-12-01
```

A. Breast-cancer

This data set includes 201 instances of one class and 85 instances of another class. The instances are described by 9 attributes, some of which are linear and some are nominal.

Number of Instances: 286
Number of Attributes: 9 + the class attribute

Attribute Information:

1. Class: no-recurrence-events, recurrence-events
2. Age: 10-19, 20-29, 30-39, 40-49, 50-59, 60-69, 70-79, 80-89, 90-99.
3. Menopause: lt40, ge40, premeno.
4. Tumor-size: 0-4, 5-9, 10-14, 15-19, 20-24, 25-29, 30-34, 35-39, 40-44, 45-49, 50-54, 55-59.
5. Inv-nodes: 0-2, 3-5, 6-8, 9-11, 12-14, 15-17, 18-20, 21-23, 24-26, 27-29, 30-32, 33-35, 36-39.
6. Node-caps: yes, no.
7. Deg-malig: 1, 2, 3.
8. Breast: left, right.
9. Breast-quad: left-up, left-low, right-up, right-low, central.
10. Birradiat: yes, no.

Missing Attribute Values: (denoted by "?")

Attribute #: Number of instances with missing values 8

Class Distribution:

1. no-recurrence-events: 201 instances
2. recurrence-events: 85 instances

Num Instances: 286
Num Attributes: 10
Num Continuous: 0 (Int 0 / Real 0)
Num Discrete: 10
Missing values: 9/0.3

Name	Type	Enum	Ints	Real	Missing		Distinct (1)
'Age'	Enum	100	0	0	0 / 0	6 / 2	0
menopause'	Enum	100	0	0	0 / 0	3 / 1	0
'tumor-size'	Enum	100	0	0	0 / 0	11 / 4	0
'inv-nodes'	Enum	100	0	0	0 / 0	7 / 2	0
'node-caps'	Enum	97	0	0	8 / 3	2 / 1	0
'deg-malig'	Enum	100	0	0	0 / 0	3 / 1	0
'breast'	Enum	100	0	0	0 / 0	2 / 1	0
'breast-quad'	Enum	100	0	0	1 / 0	5 / 2	0
'irradiat'	Enum	100	0	0	0 / 0	2 / 1	0
'Class'	Enum	100	0	0	0 / 0	2 / 1	0

@relation breast-cancer
@attribute age {'10-19','20-29','30-39','40-49','50-59','60-69','70-79','80-89','90-99'}
@attribute menopause {'lt40','ge40','premeno'}
@attribute tumor-size {'0-4','5-9','10-14','15-19','20-24','25-29','30-34','35-39','40-44','45-49','50-54','55-59'}
@attribute inv-nodes {'0-2','3-5','6-8','9-11','12-14','15-17','18-20','21-23','24-26','27-29','30-32','33-35','36-39'}
@attribute node-caps {'yes','no'}
@attribute deg-malig {'1','2','3'}

@attribute breast {'left','right'}
@attribute breast-quad {'left_up','left_low','right_up','right_low','central'}
@attribute 'irradiat' {'yes','no'}
@attribute 'Class' {'no-recurrence-events','recurrence-events'}
@data
'40-49','premeno','15-19','0-2','yes','3','right','left_up','no','recurrence-events'
'50-59','ge40','15-19','0-2','no','1','right','central','no','no-recurrence-events'
'50-59','ge40','35-39','0-2','no','2','left','left_low','no','recurrence-events'
'40-49','premeno','35-39','0-

B. Contact-lenses

Title: Database for fitting contact lenses
Sources:
Cendrowska, J. "PRISM: An algorithm for inducing modular rules", International Journal of Man-Machine Studies, 1987, 27, 349-370
Donor: Benoit Julien (Julien@ce.cmu.edu)
Date: 1 August 1990

Past Usage:
See above.
Witten, I. H. & MacDonald, B. A. (1988). Using concept learning for knowledge acquisition. International Journal of Man-Machine Studies, 27, (pp. 349-370).
Notes: This database is complete (all possible combinations of attribute-value pairs are represented). Each instance is complete and correct. 9 rules cover the training set.

Relevant Information Paragraph:
The examples are complete and noise free.
The examples highly simplified the problem. The attributes do not fully describe all the factors affecting the decision as to which type, if any, to fit.

Number of Instances: 24
Number of Attributes: 4 (all nominal)
Attribute Information:
-- 3 Classes
1 : the patient should be fitted with hard contact lenses, 2 : the patient should be fitted with soft contact lenses, 1 : the patient should not be fitted with contact lenses.

1. age of the patient: (1) young, (2) pre-presbyopic, (3) presbyopic
2. spectacle prescription: (1) myope, (2) hypermetrope
3. astigmatic: (1) no, (2) yes
4. tear production rate: (1) reduced, (2) normal

Number of Missing Attribute Values: 0
Class Distribution:
1. hard contact lenses: 4
2. soft contact lenses: 5
3. no contact lenses: 15

@relation contact-lenses
@attribute age {young, pre-presbyopic, presbyopic}
@attribute spectacle-prescrip {myope, hypermetrope}
@attribute astigmatism {no, yes}
@attribute tear-prod-rate {reduced, normal}
@attribute contact-lenses {soft, hard, none}

@data
24 instances
young,myope,no,reduced,none
young,myope,no,normal,soft
young,myope,yes,reduced,none
young,myope,yes,normal,hard
young,hypermetrope,no,reduced,none
young,hypermetrope,no,normal,soft
young,hypermetrope,yes,reduced,none
young,hypermetrope,yes,normal,hard
pre-presbyopic,myope,no,reduced,none
pre-presbyopic,myope,no,normal,soft
pre-presbyopic,myope,yes,reduced,none
pre-presbyopic,myope,yes,normal,hard
pre-presbyopic,hypermetrope,no,reduced,none
pre-presbyopic,hypermetrope,no,normal,soft
pre-presbyopic,hypermetrope,yes,reduced,none
pre-presbyopic,hypermetrope,yes,normal,none
presbyopic,myope,no,reduced,none
presbyopic,myope,no,normal,none
presbyopic,myope,yes,reduced,none
presbyopic,myope,yes,normal,hard
presbyopic,hypermetrope,no,reduced,none
presbyopic,hypermetrope,no,normal,soft
presbyopic,hypermetrope,yes,reduced,none
presbyopic,hypermetrope,yes,normal,none

C. CPU

Deleted "vendor" attribute to make data consistent with with what we used in the data mining book.
@relation 'cpu'
@attribute MYCT numeric
@attribute MMIN numeric
@attribute MMAX numeric
@attribute CACH numeric

@attribute CHMIN numeric
@attribute CHMAX numeric
@attribute class numeric

@data
125,256,6000,256,16,128,198
29,8000,32000,32,8,32,269
29,8000,32000,32,8,32,220
29,8000,32000,32,8,32,172
29,8000,16000,32,8,16,132
26,8000,32000,64,8,32,318
23,16000,32000,64,16,32,367
23,16000,32000,64,16,32,489
23,16000,64000,64,16,32,636
23,32000,64000,128,32,64,1144

D. Credit-g

Description of the German credit dataset.
Title: German Credit data
Source Information
Number of Instances: 1000

Two datasets are provided. the original dataset, in the form provided by Prof. Hofmann, contains categorical/symbolic attributes andis in the file "german.data".

For algorithms that need numerical attributes, Strathclyde University produced the file "german.data-numeric". This file has been edited and several indicator variables added to make it suitable for algorithms which cannot cope with categorical variables. Several attributes that are ordered categorical (such as attribute 17) have been coded as integer. This was the form used by StatLog.

Number of Attributes german: 20 (7 numerical, 13 categorical)
Number of Attributes german.numer: 24 (24 numerical)
Attribute description for german
Attribute 1:qualitative

Status of existing checking account
A11: ... <0 DM
A12: 0 <= ... < 200 DM
A13: ... >= 200 DM /salary assignments for at least
1 year
A14: no checking account

Attribute 2: (numerical)
Duration in month

Attribute 3: (qualitative)
Credit history

A30: no credits taken/ all credits paid back duly
A31: all credits at this bank paid back duly
A32: existing credits paid back duly till now
A33: delay in paying off in the past
A34: critical account/ other credits existing (not at this bank)

Attribute 4: (qualitative)
Purpose
A40: car (new)
A41: car (used)
A42: furniture/equipment
A43: radio/television

Attribute 5: (numerical)
Credit amount

Attibute 6: (qualitative)
Savings account/bonds
A61: ... < 100 DM
A62: 100 <= ... < 500 DM
A63: 500 <= ... < 1000 DM
A64: .. >= 1000 DM
A65: unknown/ no savings account

Attribute 7: (qualitative)
Present employment since
A71: unemployed
A72: ... < 1 year
A73: 1 <= ... < 4 years
A74: 4 <= ... < 7 years
A75: .. >= 7 years

Attribute 8: (numerical)
Installment rate in percentage of disposable income

Attribute 9: (qualitative)
Personal status and sex
A91: male: divorced/separated

A92: female: divorced/separated/married
A93: male: single
A94: male: married/widowed
A95: female: single

Attribute 10: (qualitative)
Other debtors / guarantors
A101: none
A102: co-applicant
A103: guarantor

Attribute 11: (numerical)
Present residence since

Attribute 12: (qualitative)
Property
A121: real estate
A122: if not A121: building society savings agreement/life insurance

Attribute 13: (numerical)
Age in years

Attribute 14: (qualitative)
Other installment plans
A141: bank
A142: stores
A143: none

Attribute 15: (qualitative)
Housing
A151: rent
A152: own
A153: for free

Attribute 16: (numerical)
Number of existing credits at this bank

Attribute 17: (qualitative)
Job
A171: unemployed/ unskilled - non-resident
A172: unskilled - resident
A173: skilled employee / official
A174: management/ self-employed/ highly qualified employee/ officer

Attribute 18: (numerical)
Number of people being liable to provide maintenance for

Attribute 19: (qualitative)
Telephone
A191: none
A192: yes, registered under the customers name

Attribute 20: (qualitative)
Foreign worker
A201: yes
A202: no

Cost Matrix: This dataset requires use of a cost matrix (see below)

	1	2
1	0	2
2	5	0

(1 = Good, 2 = Bad)

The rows represent the actual classification and the columns the predicted classification. It is worse to class a customer as good when they are bad (5), than it is to class a customer as bad when they are good (1).

Relabeled values in attribute checking_status

From: A11 To: '<0'
From: A12 To: '0<=X<200'
From: A13 To: '>=200'
From: A14 To: 'no checking'

@relation german_credit
@attribute checking_status {'<0', '0<=X<200', '>=200', 'no checking'}
@attribute duration numeric
@attribute credit_history {'no credits/all paid', 'all paid', 'existing paid', 'delayed previously', 'critical/other existing credit'}
@attribute purpose {'new car', 'used car', furniture/equipment, radio/tv, 'domestic appliance', repairs, education, vacation, retraining, business, other}
@attribute credit_amount numeric
@attribute savings_status {'<100', '100<=X<500', '500<=X<1000', '>=1000', 'no known savings'}
@attribute employment {unemployed, '<1', '1<=X<4', '4<=X<7', '>=7'}
@attribute installment_commitment numeric
@attribute personal_status { 'male div/sep', 'female div/dep/mar', 'male single', 'male mar/wid', 'female single'}
@attribute other_parties {none, 'co applicant', guarantor}
@attribute residence_since numeric
@attribute property_magnitude {'real estate', 'life insurance', car, 'no known property'}
@attribute age numeric
@attribute other_payment_plans {bank, stores, none}
@attribute housing {rent, own, 'for free'}
@attribute existing_credits numeric
@attribute job {'unemp/unskilled non res', 'unskilled resident', skilled, 'high qualif/self emp/mgmt'}
@attribute num_dependents numeric
@attribute own_telephone {none, yes}
@attribute foreign_worker {yes, no}
@attribute class {good, bad}
@data
'<0',6,'critical/other existing credit',radio/tv,1169,'no known savings','>=7',4,'male single',none,4,'real estate',67,none,own,2,skilled,1,yes,yes,good
'0<=X<200',48,'existing paid',radio/tv,5951,'<100',
'1<=X<4',2,'female div/dep/mar',none,2,'real estate',22,none,own,1,skilled,1,none,yes,bad 'no checking',12,'critical/other existing credit',education,2096,'<100',

'4<=X<7',2,'male single',none,3,'real estate',49,none,own,1,'unskilled resident',2,none,yes,good '<0',42,'existing paid',furniture/equipment,7882,'<100',
'4<=X<7',2,'male single',guarantor,4,'life insurance',45,none,'for free',1,skilled,2,none,yes,good '<0',24,'delayed previously','new car',4870,'<100',
'1<=X<4',3,'male single',none,4,'no known property',53,none,'for free',2,skilled,2,none,yes,bad 'no checking',36,'existing paid',education,9055,'no known savings',
'1<=X<4',2,'male single',none,4,'no known property',35,none,'for free',1,'unskilled resident',2,yes,yes,good 'no checking',24,'existing paid',furniture/equipment,2835,
'500<=X<1000','>=7',3,'male single',none,4,'life insurance',53,none,own,1,skilled,1,none,yes,good '0<=X<200',36,'existing paid','used car',6948,'<100',
'1<=X<4',2,'male single',none,2,car,35,none,rent,1,'high qualif/self emp/mgmt',1,yes,yes,good.

VIVA VOICE QUESTIONS

Q1: What is a database?

A1: A database is a structured collection of data that is organized and stored in a way that allows for efficient storage, retrieval, and management of data. It provides a systematic way to store and manage large amounts of structured information.

Q2: What is the purpose of a primary key in a database table?

A2: A primary key is a unique identifier for each record (row) in a database table. It ensures that each record has a unique identity and allows for efficient retrieval and referencing of specific records. The primary key constraint enforces the uniqueness and integrity of the data.

Q3: In machine learning, what are attributes?

A3: In machine learning, attributes refer to the features or characteristics of a data instance. They represent the different properties or variables associated with each data point. Attributes can be numerical (e.g., age, income) or categorical (e.g., gender, color), and they are used as inputs to machine learning algorithms to make predictions or classifications.

Q4: What is the role of data normalization in machine learning?

A4: Data normalization is a preprocessing step in machine learning that scales the attribute values to a common range. It ensures that all attributes contribute equally to the learning process by preventing attributes with larger value ranges from dominating the algorithm. Normalization helps improve the performance and convergence of machine learning models.

Q5: What is the difference between structured and unstructured data?

A5: Structured data refers to data that is organized and follows a predefined schema or data model, such as data stored in databases with well-defined tables and relationships. Unstructured data, on the other hand, does not have a predefined structure and lacks a fixed schema. Examples of unstructured data include text documents, images, audio files, and social media posts. Machine learning techniques often require structured data or need preprocessing steps to handle unstructured data.

EXPERIMENT – 5

AIM - Working of Major Classifiers:

a) Naïve Bayes b) Decision Tree c) CART d) ARIMA e) linear and logistics regression f) Support vector machine g) KNN

(a) Naive Bayes algorithm

Using the Python library scikit-learn to build the Naive Bayes algorithm.

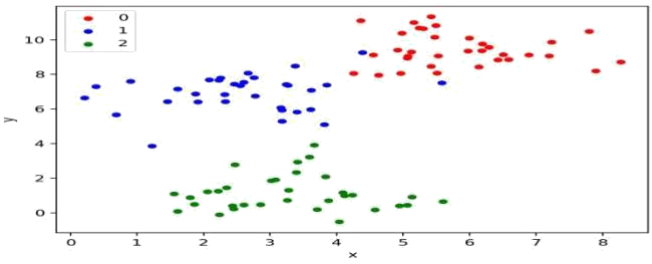
```
>>>     from sklearn.naive_bayes import GaussianNB
>>>     from sklearn.naive_bayes import MultinomialNB
>>>     from sklearn import datasets
>>>     from sklearn.metrics import confusion_matrix
>>>     iris = datasets.load_iris()
>>>     gnb = GaussianNB()
>>>     mnb = MultinomialNB()
>>>     y_pred_gnb = gnb.fit(iris.data, iris.target).predict(iris.data)
>>>     cnf_matrix_gnb = confusion_matrix(iris.target, y_pred_gnb)
>>>     print(cnf_matrix_gnb)
```

```
[[50 0 0]
[047 3]
[0 347]]
```

```
>>>     y_pred_mnb = mnb.fit(iris.data, iris.target).predict(iris.data)
>>>     cnf_matrix_mnb = confusion_matrix(iris.target, y_pred_mnb)
>>>     print(cnf_matrix_mnb)
```

```
[[50 0 0]
[046 4]
[0 347]]
```

Output:



b) Decision Tree

```
import numpy as np import
pandas as pd
from sklearn.metrics import confusion_matrix from
```

```
sklearn.cross_validation import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

#Function importing Dataset
def importdata():
 balance_data = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-'+
'databases/balance-scale/balance-scale.data', sep= ',', header = None)

# Printing the dataset shape
print("Dataset Length: ", len(balance_data))
print ("Dataset Shape: ", balance_data.shape)

#Printing the dataset obseravtions
print ("Dataset: ",balance_data.head())
return balance_data

#Function to split the dataset
def splitdataset(balance_data):

#Separating the target variable

X = balance_data.values[:, 1:5]

Y = balance_data.values[:, 0]

# Splitting the dataset into train and test

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.3, random_state = 100)

return X, Y, X_train, X_test, y_train, y_test

#Function to perform training with giniIndex.
 def train_using_gini(X_train, X_test, y_train):

# Creating the classifier object

clf_gini = DecisionTreeClassifier(criterion = "gini", random_state = 100,max_depth=3,
min_samples_leaf=5)

# Performing training

clf_gini.fit(X_train, y_train)
return clf_gini
```

```
#Function to perform training with entropy.
def tarin_using_entropy(X_train, X_test, y_train):

# Decision tree with entropy
DecisionTreeClassifier(clf_entropy = criterion = "entropy", random_state = 100, max_depth
= 3, min_samples_leaf = 5)

# Performing training

clf_entropy.fit(X_train, y_train)
return clf_entropy

#Function to make predictions
def prediction(X_test, clf_object):

#Predicton on test with giniIndex
y_pred = clf_object.predict(X_test)
print("Predicted values:")
print(y_pred)
return y_pred

#Function to calculate accuracy
def cal_accuracy(y_test, y_pred):
print("Confusion Matrix: ", confusion_matrix(y_test, y_pred))
print ("Accuracy : ", accuracy_score(y_test,y_pred)*100)

print("Report : ", classification_report(y_test, y_pred))

#Driver code
def main():
#Building Phase
data = importdata()
X, Y, X_train, X_test, y_train, y_test = splitdataset(data)
clf_gini = train_using_gini(X_train, X_test, y_train)
clf_entropy = tarin_using_entropy(X_train, X_test, y_train)

#Operational Phase

print("Results Using Gini Index:")

# Prediction using gini y_
pred_gini =prediction(X_test, clf_gini)

cal_accuracy(y_test, y_pred_gini)
```

```
print("Results Using Entropy:")

# Prediction using entropy

y_pred_entropy = prediction(X_test, clf_entropy)

cal_accuracy(y_test, y_pred_entropy)

#Calling main function
if __name__=="__main__": main()
```

Output:

```
Predicted values:
['R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'L'
'L' 'R' 'L' 'R' 'L' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'L' 'L'
'L' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'R' 'L' 'L' 'R' 'L' 'L' 'R' 'L' 'L'
'R' 'L' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'L' 'R' 'L' 'L' 'L' 'R'
'R' 'L' 'R' 'L' 'R' 'R' 'R' 'L' 'R' 'L' 'L' 'L' 'L' 'R' 'R' 'L' 'R' 'L'
'R' 'R' 'L' 'L' 'L' 'R' 'R' 'L' 'L' 'L' 'R' 'L' 'L' 'R' 'R' 'R' 'R' 'R'
'R' 'L' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L'
'L' 'L' 'L' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'L' 'R'
'L' 'R' 'R' 'L' 'L' 'R' 'L' 'R' 'R' 'R' 'R' 'R' 'L' 'R' 'R' 'R' 'R' 'R'
'R' 'L' 'R' 'L' 'R' 'R' 'L' 'R' 'L' 'R' 'L' 'R' 'L' 'L' 'L' 'L' 'L' 'R'
'R' 'R' 'L' 'L' 'L' 'R' 'R' 'R']

Confusion Matrix:  [[ 0  6    7]
                    [06322]
                   [ 0 20 70]]
Accuracy :  70.7446808511
Report :
        precision    recall f1-score     support
   B        0.00     0.00       0.00        13
L       0.71     0.74    0.72       85      R
0.71       0.78      0.74        90 avg / total
0.66     0.71       0.68    188
```

c) CART

```
#CART on the Bank Note dataset from random import seed from

random import randrange from

csv import reader

# Load a CSV file
Def load_csv(filename):
file = open(filename, "rt")
lines = reader(file)
dataset = list(lines)
return dataset

#Convert string column to float

def str_column_to_float(dataset, column):
for row in dataset:
row[column] = float(row[column].strip())

#Split a dataset into k folds
Def cross_validation_split(dataset, n_folds):

dataset_split = list()
dataset_copy = list(dataset)
fold_size = int(len(dataset) / n_folds)
for i in range(n_folds):
fold = list()
while len(fold) < fold_size:
index = randrange(len(dataset_copy))
fold.append(dataset_copy.pop(index))
dataset_split.append(fold)
return dataset_split



#Calculate accuracy percentage
def accuracy_metric(actual, predicted):
correct = 0
for i in range(len(actual)):
if actual[i] == predicted[i]:
correct += 1
return correct / float(len(actual)) * 100.0

#Evaluate an algorithm using a cross validation split
```

```
def evaluate_algorithm(dataset, algorithm, n_folds, *args):
folds = cross_validation_split(dataset, n_folds)

scores = list()
for fold in folds:
train_set = list(folds)

train_set.remove(fold)
train_set = sum(train_set, [])
test_set = list()
for row in fold:
row_copy = list(row)
test_set.append(row_copy)

row_copy[-1] = None

predicted = algorithm(train_set, test_set, *args)

actual = [row[-1] for row in fold]

accuracy = accuracy_metric(actual, predicted)

scores.append(accuracy)
return scores



#Split a dataset based on an attribute and an attribute value
def test_split(index, value, dataset):
left, right = list(), list()
for row in dataset:
if row[index] < value:
left.append(row)
else:
right.append(row)
return left, right

# Calculate the Gini index for a split dataset

  def gini_index(groups, classes):

# count all samples at split point

  n_instances = float(sum([len(group) for group in groups]))
```

d) ARIMA

```
import pandas as pd
data = pd.read_csv(“Electric_Production.csv”,index_col=0)
data.head() data.index = pd.to_datetime(data.index)
data.columns = ['Energy Production']


import plotly.plotly as ply
import cufflinks as cfdata.iplot(title="Energy Production Jan 1985--Jan 2018")

from plotly.plotly
import plot_mpl from statsmodels.tsa.seasonal
import seasonal_decompose
result = seasonal_decompose(data, model=’multiplicative’)
fig = result.plot() plot_mpl(fig)

from pyramid.arima import auto_arimastepwise_model =
auto_arima(data, start_p=1, start_q=1, max_p=3, max_q=3, m=12,
start_P=0, seasonal=True, d=1, D=1, trace=True,
error_action='ignore', suppress_warnings=True,
stepwise=True)print(stepwise_model.aic())
```

Running the example prints a summary of the fit model. This summarizes the coefficient values used as well as the skill of the fit on the on the in-sample observations.

```
                             ARIMA Model Results

==================================================================
===== ==
```

```
1
2
3
```

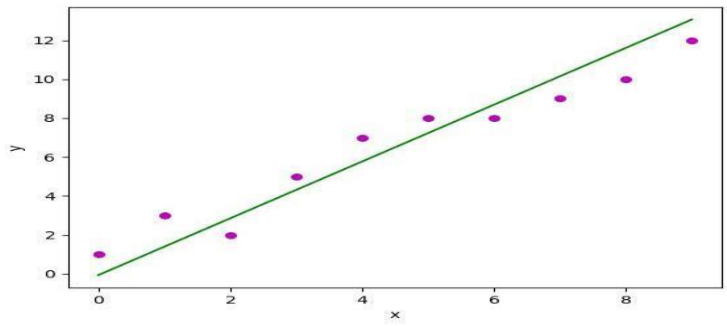
```
Dep. Variable:                 D.Sales No. Observations:                   35
Model:              ARIMA(5, 1, 0)  Log Likelihood                 -196.170
Method:                    css-mle S.D. of innovations              64.241
Date:                                          Mon, 12 Dec 2016          AIC
406.340

Time:                                                   11:09:13        BIC
417.227

Sample:                     02-01-1901 HQIC
410.098

                           - 12-01-1903

==================================================================================
                     coef    std err          z      P>|z|      [95.0% Conf.
Int.]
```

	coef	std err	z	P>|z|	[95.0% Conf. Int.]
const	12.0649	3.652	3.304	0.003	4.908 19.222
ar.L1.D.Sales	-1.1082	0.183	-6.063	0.000	-1.466 -0.750
ar.L2.D.Sales	-0.6203	0.282	-2.203	0.036	-1.172 -0.068

e) Linear Regression:

```
import numpy as np

import matplotlib.pyplot as plt

def estimate_coef(x, y):

#number of observations/points
 n = np.size(x)

#mean of x and y vector

m_x, m_y = np.mean(x), np.mean(y)
```

```
#calculating cross-deviation and deviation about x
SS_xy = np.sum(y*x) - n*m_y*m_x

SS_xx = np.sum(x*x) - n*m_x*m_x

#calculating regression coefficients

b_1 = SS_xy / SS_xx

b_0 = m_y - b_1*m_x

return(b_0, b_1)

def plot_regression_line(x, y, b):
#plotting the actual points as scatter plot

plt.scatter(x, y, color = "m", marker = "o", s = 30)
# predicted response vector

y_pred = b[0] + b[1]*x
#plotting the regression line plt.plot(x, y_pred, color = "g")

# putting labels

plt.xlabel('x')     plt.ylabel('y')

#function to show plot plt.show()

def main():
# observations
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

# estimating coefficients
b = estimate_coef(x, y)
print("Estimated coefficients:\nb_0 = {} \ \nb_1 = {}".format(b[0], b[1]))

#plotting regression line
plot_regression_line(x, y, b)

if __name__ == "__main__":
main()
```



VIVA VOICE QUESTIONS

Q1: What is a decision tree classifier?
A1: A decision tree classifier is a supervised machine learning algorithm that uses a tree-like model to make predictions. It splits the input space into regions based on the feature values and creates a tree structure where each internal node represents a decision based on a feature, and each leaf node represents a class label or outcome. Decision trees are easy to interpret and can handle both categorical and numerical data.

Q2: What is a support vector machine (SVM) classifier?
A2: A support vector machine classifier is a supervised machine learning algorithm used for binary classification and can be extended to handle multi-class problems. SVM constructs a hyperplane or set of hyperplanes in a high-dimensional space that separates the different classes with the maximum margin. It aims to find the optimal decision boundary that maximizes the distance between the support vectors and the hyperplane.

Q3: What is a random forest classifier?
A3: A random forest classifier is an ensemble learning method that combines multiple decision trees to make predictions. It works by creating a set of decision trees on randomly sampled subsets of the training data and aggregating the predictions of individual trees to make the final prediction. Random forests are robust, handle high-dimensional data well, and can capture complex relationships between features.

Q4: What is a logistic regression classifier?
A4: Logistic regression is a supervised machine learning algorithm used for binary classification. Despite its name, logistic regression is a classification algorithm, not a regression algorithm. It models the relationship between the input features and the probability of the binary outcome using a logistic function. Logistic regression is widely used due to its simplicity, interpretability, and effectiveness in a variety of applications.

Q5: What is a k-nearest neighbors (KNN) classifier?
A5: The k-nearest neighbors classifier is a supervised machine learning algorithm used for both classification and regression tasks. It classifies a new data point based on the majority class label of its k nearest neighbors in the feature space. KNN does not build an explicit model but relies on the training instances for classification. It is simple to implement and can handle both numerical and categorical data.

EXPERIMENT - 6

AIM - Design a prediction model for Analysis of round trip Time of Flight measurement from asupermarket using random forest, naïve bayes etc.

CMU-SuperMarket Data Collection and Preprocessing for round-trip time of Flight(RToF) .

```
import pandas as pd
import numpy as np from sklearn.model_selection
import train_test_split from sklearn.metrics
import mean_squared_error as mse
data = pd.read_csv("cmu_supermarket.csv")
data = data.iloc[: , 0:5] data.columns = ['X', 'Y','Z','Mag','RToF'] data = data.dropna()
data.head()
```

	X	Y	Z	Mag	RToF
0	5	0	0	3167.0	17.44
1	5	0	0	3218.0	18.49
4	5	0	0	3183.0	15.02
6	5	0	0	3160.0	16.74
7	5	0	0	3161.0	14.94

```
X = data.iloc[: , 0:4]
Y = data.iloc[: , 4]
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.10, random_state=33)
```

Algorithms:

1. Naïve Bayes

```
from sklearn.linear_model import BayesianRidge
nb_model = BayesianRidge()
nb_model = nb_model.fit(X_train , Y_train)
prediction = nb_model.predict(X_test)
rmse = (mse(y_true = Y_test , y_pred = prediction))**1/2 print("RMSE : " , rmse)
```

```
RMSE  :  9.127520132505145
```

2) Decision Tree

```
From sklearn.tree import DecisionTreeRegressor
dt_model = DecisionTreeRegressor()
```

```
dt_model = dt_model.fit(X_train , Y_train)
prediction = dt_model.predict(X_test)
rmse = (mse(y_true = Y_test , y_pred = prediction))**1/2
print("RMSE : " , rmse)
```

```
RMSE  :  2.214809487951807
```

3) Random Forest

```
from  sklearn.ensemble  import RandomForest
Regressorrf_model = RandomForestRegressor()
rf_model= rf_model.fit(X_train , Y_train)
prediction = rf_model.predict(X_test)
rmse = (mse(y_true = Y_test , y_pred = prediction))**1/2
print("RMSE : " , rmse)
```

```
RMSE  :  1.4735218540718842
```

4) SVM

```
from sklearn.svm import SVR
svm_model = SVR()
svm_model = svm_model.fit(X_train , Y_train)
prediction = svm_model.predict(X_test)
rmse = (mse(y_true = Y_test , y_pred=prediction))**1/2
print("RMSE : " , rmse)
```

```
RMSE  :  14.82720182239393
```

7. KNN

```
from sklearn.neighbors import KNeighborsRegressor
knn_model = KNeighborsRegressor(n_neighbors=5)
knn_model = knn_model.fit(X_train , Y_train)
prediction = knn_model.predict(X_test)
rmse = (mse(y_true = Y_test , y_pred = prediction))**1/2
print("RMSE : " , rmse)
```

```
RMSE  :  2.428723668674699
```

Result:

By performing the regression using the above five algorithms, we observe that 'Random Forest' is most accurate with least Root Mean Absolute Error.

Therefore, we conclude that out of the above algorithms, Random Forest performs best.

VIVA VOICE QUESTIONS

Q. What is ensemble learning?

Ans. To solve a particular computational program, multiple models such as classifiers or experts are strategically generated and combined. This process is known as ensemble learning.

Q. Why ensemble learning is used?

Ans. Ensemble learning is used to improve the classification, prediction, function approximation etc of a model.

Q. When to use ensemble learning?

Ans. Ensemble learning is used when you build component classifiers that are more accurate and independent from each other.

Q. What are the two paradigms of ensemble methods?

Ans. The two paradigms of ensemble methods are:

- Sequential ensemble methods
- Parallel ensemble methods

EXPERIMENT - 7

AIM - Implement supervised learning (KNN classification) .Estimate the accuracy of using 5-fold cross-validation. Choose the appropriate options for missing values.

SAMPLE INPUT:

STEP 1: OPEN WEKA GUI AND SELECT EXPLORER
STEP 2: LOAD THE BREAST CANCER DATA SET (BREAST-CANCER.ARFF) GIVEN IN EXPERIMENT 4 DATABASES, USING "OPEN FILE"
STEP 3: CHOOSE THE "CLASSIFY" TAB AND CHOOSE THE DECISION TREE CLASSIFIER LABELLED AS *J48* IN THE *TREES* FOLDER AND CHOOSE "CROSS-VALIDATION" AND ENTER *5* IN THE FOLDS, THEN PRESS *START*

OUTPUT:

```
=== RUN INFORMATION ===

SCHEME:         WEKA.CLASSIFIERS.TREES.J48 -C 0.25 -M
2 RELATION:  BREAST-CANCERINSTANCES: 286 ATTRIBUTES:  10
AGE MENOPAUSE TUMOR-SIZE
INV-NODESNODE-CAPS
DEG-MALIG BREAST BREAST-QUADIRRADIAT
CLASS
TEST MODE:       5-FOLD CROSS-VALIDATION

=== CLASSIFIER MODEL (FULL TRAINING SET)

=== J48 PRUNED TREE
  ------------

NODE-CAPS = YES
|   DEG-MALIG = 1: RECURRENCE-EVENTS (1.01/0.4)
|   DEG-MALIG = 2: NO-RECURRENCE-EVENTS (26.2/8.0)
|   DEG-MALIG = 3: RECURRENCE-EVENTS (30.4/7.4)
NODE-CAPS = NO: NO-RECURRENCE-EVENTS

(228.39/53.4) NUMBER OF LEAVES  :       4
```

```
SIZE OF THE TREE :         6


TIME TAKEN TO BUILD MODEL: 0.02 SECONDS

=== STRATIFIED CROSS-VALIDATION ===
  == SUMMARY ===

  ORRECTLY CLASSIFIED INSTANCES         12               74.1259 %
  NCORRECTLY CLASSIFIED INSTANCES       4                25.8741 %
  APPA STATISTIC                           0.2288
  EAN ABSOLUTE ERROR                       0.3726
  OOT MEAN SQUARED ERROR                   0.4435
  ELATIVE ABSOLUTE ERROR                  89.0412
  OOT RELATIVE SQUARED ERROR              97.0395
  OTAL NUMBER OF INSTANCES             86

  == DETAILED ACCURACY BY CLASS   ==

               TP RATE  FP RATE  PRECISION  RECALL     F-MEASURE  MCC      ROC
AREA  PRC AREA  CLASS
          .960            0.776    0.745     0.960    .839        0.287
  .582    .728        NO-RECURRENCE-EVENTS
          .224            0.040    0.704     0.224    .339        0.287
  .582    .444        RECURRENCE-EVENTS
  EIGHTED  VG.      0.741    0.558    0.733     0.741    .691        0.287
  .582    .643

=== CONFUSION MATRIX ===

A        B   <-- CLASSIFIED AS
193   8 |   A = NO-RECURRENCE-EVENTS
66  19 |   B = RECURRENCE-EVENTS
```

VIVA VOICE QUESTIONS

Q1: What is supervised learning?
A1: Supervised learning is a machine learning technique where the algorithm learns from labeled training data. It involves training a model on input-output pairs, where the input data is accompanied by the corresponding output labels. The model learns to predict the output labels for new, unseen data based on the patterns observed in the labeled training data.

Q2: What is the K-Nearest Neighbors (KNN) classification algorithm?
A2: The K-Nearest Neighbors (KNN) classification algorithm is a supervised machine learning algorithm used for classification tasks. It classifies a new data point based on the majority class label of its K nearest neighbors in the feature space. The value of K determines the number of neighbors to consider. KNN is a non-parametric algorithm, meaning it does not make any assumptions about the underlying data distribution.

Q3: How does the KNN algorithm determine the class label of a new data point?
A3: The KNN algorithm determines the class label of a new data point by considering the class labels of its K nearest neighbors. It computes the distance between the new data point and all the training data points using a distance metric (e.g., Euclidean distance). The K nearest neighbors with the shortest distances are identified, and the majority class label among them is assigned to the new data point.

Q4: What are the advantages of the KNN algorithm?
A4: Some advantages of the KNN algorithm include

- Simplicity and ease of implementation.
- No assumptions about the underlying data distribution.
- Ability to handle both numerical and categorical data.
- Flexibility in choosing the value of K to balance between bias and variance.
- Capability to adapt to new data points and handle incremental learning.

Q5: What are the limitations of the KNN algorithm?
A5: Some limitations of the KNN algorithm include:

- Computational complexity increases as the size of the training data grows.
- Sensitivity to the choice of distance metric and the value of K.
- The need for proper feature scaling to ensure all features contribute equally.
- Difficulty in handling datasets with imbalanced class distributions.
- Inefficiency in high-dimensional feature spaces (curse of dimensionality)

.

EXPERIMENT - 8

AIM - Introduction to R. Be aware of basics of machine learning methods in R.

R is a powerful programming language and software environment widely used for statistical computing and data analysis. This document aims to provide an introduction to R and its basics, including data types, variables, operators, and basic data manipulation operations. By the end of this tutorial, you will have a solid foundation in R programming and be ready to explore more advanced concepts.

R Environment:

The R programming environment provides an interactive interface for executing R code. It consists of a console, where commands are entered and executed, and an editor for writing and saving scripts. Familiarize yourself with the R environment and its various components to get started.

Variables and Data Types:

In R, variables are used to store data. R supports various data types, including numeric, character, logical, and factor. Learn how to assign values to variables using the assignment operator, and perform basic arithmetic operations using variables.

Data Input and Output:

Data can be read into R from various sources, such as CSV files or databases. Explore the read.csv() function to load data into R. Create a sample dataset manually or load an existing dataset to understand its structure and contents. Practice saving datasets in different formats using suitable R functions.

Data Manipulation:

Data manipulation is a crucial aspect of data analysis. Learn how to subset, filter, and sort data using R's functions. Familiarize yourself with operations on data frames, such as subsetting with logical conditions, filtering rows based on specific criteria, and sorting data by specific columns. Perform basic data transformations, such as adding new columns or modifying existing ones.

Summary Statistics and Visualization:

R provides various functions for calculating summary statistics, such as mean, median, minimum, maximum, and summary. Explore these functions to analyze numeric variables in your dataset.

Additionally, R offers built-in plotting functions for creating visualizations. Practice creating simple plots, histograms, and boxplots to gain insights into your data.

Machine Learning Methods in R

R has a vast array of machine learning algorithms and packages for various tasks. Gain familiarity with popular machine learning methods available in R, such as:

1. Linear Regression:
   - Fit a linear model to predict numeric outcomes based on a set of predictor variables.
   - Use the lm() function to perform linear regression analysis.
2. Decision Trees:
   - Construct tree-based models for classification and regression tasks using packages like rpart or randomForest.
   - Use the rpart() function to build a decision tree model.
3. Logistic Regression:
   - Perform binary or multinomial classification using the logistic regression algorithm.
   - Use the glm() function with the family argument set to "binomial" for logistic regression.
4. k-Nearest Neighbors (KNN):
   - Utilize the KNN algorithm for classification or regression by identifying the nearest neighbors in the feature space.
   - Use the knn() function from the class package to implement KNN.
5. Support Vector Machines (SVM):
   - Apply SVM algorithms for classification or regression tasks, separating data points with hyperplanes.
   - Use the svm() function from the e1071 package to build SVM models.
6. Random Forests:
   - Build an ensemble of decision trees to perform classification or regression using the randomForest package.
   - Use the randomForest() function to create random forest models.
7. Naive Bayes:
   - Implement the Naive Bayes algorithm for classification tasks, assuming independence among features.

VIVA VOICE QUESTIONS

Q. What is Regularization?

Ans. Often called as the C parameter in python's sklearn library, the regularization parameter commands the support vector machine on the optimal amount of misclassification of each training data it wants to avoid. Such support vector machine example is when larger numbers are used for the C parameter, the optimization will automatically choose a hyper- plane margin which is smaller if it successfully gets all the training data points correctly separated and classified.

Alternatively, for really small values, it will cause the algorithm to look for a larger margin to be separated by the hyper-plane, even if the hyper-plane might misclassify some data points.

Q. What is Ensemble Learning?

Ans. Ensemble methods are meta-algorithms that combine several machine learning techniques into one predictive model in order to decrease variance (bagging), bias (boosting), or improve predictions.

Q. What is Bagging?

Ans. Bagging: Bagging stands for bootstrap aggregation. One way to reduce the variance of an estimate is to average together multiple estimates. For example, we can train M different trees on different subsets of the data (chosen randomly with replacement) and compute the ensemble:

$$f(x) = 1/M \sum_{m=1}^{M} f_m(x)$$

Bagging uses bootstrap sampling to obtain the data subsets for training the base learners. For aggregating the outputs of base learners, bagging uses voting for classification and averaging for regression.

Q. What is Boosting?

Ans. Boosting refers to a family of algorithms that are able to convert weak learners to strong learners. The main principle of boosting is to fit a sequence of weak learners− models that are only slightly better than random guessing, such as small decision trees− to weighted versions of the data. More weight is given to examples that were misclassified by earlier rounds. The predictions are then combined through a weighted majority vote (classification) or a weighted sum (regression) to produce the final prediction. The principal difference between boosting and the committee methods, such as bagging, is that base learners are trained in sequence on a weighted version of the data.

EXPERIMENT-9

AIM - Build and Develop a model in R for a particular classifier (random Forest).

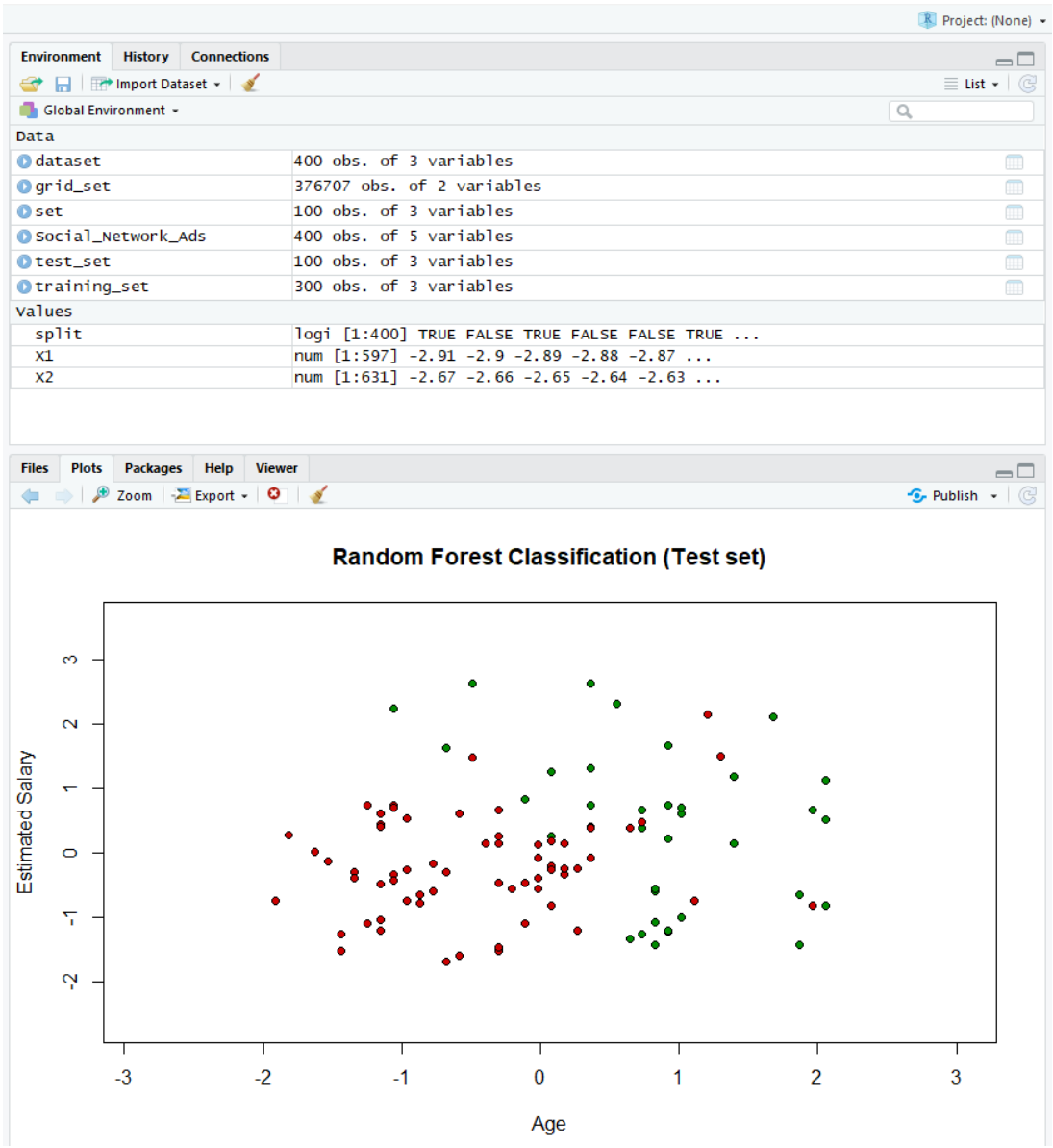
Code:

```
# Importing the dataset

dataset = read.csv('Social_Network_Ads.csv')

dataset = dataset[3:5]


# Encoding the target feature as factor

dataset$Purchased = factor(dataset$Purchased, levels = c(0, 1))


# Splitting the dataset into the Training set and Test set#
install.packages('caTools')
library(caTools)
set.seed(123)
split = sample.split(dataset$Purchased, SplitRatio = 0.75)
training_set = subset(dataset, split == TRUE)
test_set = subset(dataset, split == FALSE)


# Feature Scaling

training_set[-3] = scale(training_set[-3])

test_set[-3] = scale(test_set[-3])


# Fitting Random Forest Classification to the Training set#

install.packages('randomForest')
library(randomForest)

set.seed(123)
```

```
classifier = randomForest(x = training_set[-3], y = training_set$Purchased,ntree = 500)

# Predicting the Test set results

y_pred = predict(classifier, newdata = test_set[-3])

# Making the Confusion Matrixcm = table(test_set[, 3], y_pred)

# Visualising the Training set results

library(ElemStatLearn)
set = training_set

X1 = seq(min(set[, 1]) - 1, max(set[, 1]) + 1, by = 0.01)

X2 = seq(min(set[, 2]) - 1, max(set[, 2]) + 1, by = 0.01)

grid_set = expand.grid(X1, X2)
colnames(grid_set) = c('Age', 'EstimatedSalary')
y_grid = predict(classifier, grid_set)
plot(set[, -3], main = 'Random Forest Classification (Training set)',xlab = 'Age', ylab =
'Estimated Salary', xlim = range(X1), ylim = range(X2))

contour(X1, X2, matrix(as.numeric(y_grid), length(X1), length(X2)), add = TRUE)
points(grid_set, pch = '.', col = ifelse(y_grid == 1, 'springgreen3', 'tomato'))

points(set, pch = 21, bg = ifelse(set[, 3] == 1, 'green4', 'red3'))

# Visualising the Test set results
library(ElemStatLearn)
set = test_set
X1 = seq(min(set[, 1]) - 1, max(set[, 1]) + 1, by = 0.01)

X2 = seq(min(set[, 2]) - 1, max(set[, 2]) + 1, by = 0.01)

grid_set = expand.grid(X1, X2)
colnames(grid_set) = c('Age', 'EstimatedSalary')
y_grid = predict(classifier, grid_set)
plot(set[, -3], main = 'Random Forest Classification (Test set)',xlab = 'Age', ylab = 'Estimated
Salary', xlim = range(X1), ylim = range(X2))
```

```
contour(X1, X2, matrix(as.numeric(y_grid), length(X1), length(X2)), add = TRUE)
points(grid_set, pch = '.', col = ifelse(y_grid == 1, 'springgreen3', 'tomato'))

points(set, pch = 21, bg = ifelse(set[, 3] == 1, 'green4', 'red3'))


# Choosing the number of treesplot(classifier)
```

OUTPUT:



VIVA VOICE QUESTIONS

Q1: What is the Random Forest classifier?

A1: The Random Forest classifier is an ensemble learning method that combines multiple decision trees to make predictions. It randomly selects subsets of the training data and features for each tree and aggregates the predictions to determine the final outcome. Random Forest is robust, handles high-dimensional data well, and can capture complex relationships between features.

Q2: How does the Random Forest classifier handle overfitting?

A2: Random Forest combats overfitting by using random subsets of the training data and features for each tree. This randomness introduces diversity among the trees, reducing the correlation and overfitting. Additionally, Random Forest performs feature randomization, considering a random subset of features at each split, which further helps in reducing overfitting.

Q3: How does the Random Forest classifier handle missing data?

A3: Random Forest can handle missing data by utilizing the concept of surrogate splits. During the training process, if a value is missing for a particular feature, the algorithm calculates an alternative split using surrogate variables correlated with the missing feature. This ensures that missing data does not significantly impact the classification performance.

Q4: What are the advantages of the Random Forest classifier?

A4: Some advantages of the Random Forest classifier include:

- **a)** Ability to handle high-dimensional data and large feature spaces.
- **b)** Robustness against overfitting and resistance to noisy data.
- **c)** Interpretability through feature importance measures.
- **d)** Good performance even without extensive hyperparameter tuning.
- **e)** Capability to handle both classification and regression tasks.

Q5: Can the Random Forest classifier handle categorical variables?

A5: Yes, the Random Forest classifier can handle categorical variables. It can work with both categorical and numerical features. For categorical variables, the algorithm uses techniques like one-hot encoding or ordinal encoding to convert them into numerical representations that can be used for building decision trees.

EXPERIMENT - 10

AIM - Develop a machine learning method using Neural Networks in Python to Predict stockprices based on past price variation.

Program :

```
#Import the libraries
import  math
import  pandas_datareader  as web
import  numpy  as  np
import pandas  as  pd
from  sklearn.preprocessing
import MinMaxScaler from  keras.models
import Sequential from  keras.layers
import  Dense, LSTM
import  matplotlib.pyplot  as  plt

plt.style.use('fivethirtyeight')

#Get the stock quote

df = web.DataReader('AAPL', data_source='yahoo', start='2012-01-01', end='2019-12-17')

#Show the data
df

#Visualize the closing price history
plt.figure(figsize=(16,8))
plt.title('Close Price History')
plt.plot(df['Close'])
plt.xlabel('Date',fontsize=18)
plt.ylabel('Close Price USD ($)',fontsize=18)
plt.show()

#Create a new dataframe with only the 'Close' column

data = df.filter(['Close'])
#Converting the dataframe to a numpy array
dataset = data.values
#Get /Compute the number of rows to train the model on training_data_len = math.ceil(
len(dataset) *.8)


#Scale the all of the data to be values between 0 and 1
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
scaled_data = scaler.fit_transform(dataset)

#Create the scaled training data set

train_data = scaled_data[0:training_data_len , : ]
#Split the data into x_train and y_train data sets
x_train=[]
y_train = []
for i in range(60,len(train_data)):
x_train.append(train_data[i-60:i,0])
y_train.append(train_data[i,0])

#Convert x_train and y_train to numpy arrays
x_train, y_train = np.array(x_train),
np.array(y_train)

#Build the LSTM network model
model = Sequential()

model.add(LSTM(units=50,return_sequences=True,input_shape=(x_train.shape[1],1) ))
model.add(LSTM(units=50, return_sequences=False))

model.add(Dense(units=25))
model.add(Dense(units=1))

#Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

#Train the model
model.fit(x_train, y_train, batch_size=1, epochs=1)

#Test data set
test_data = scaled_data[training_data_len - 60: , : ]

#Create the x_test and y_test data sets
x_test = []
y_test = dataset[training_data_len : , : ]
#Get all of the rows from index 1603 to the rest and all of the columns (in this case it's only
column 'Close'), so 2003 - 1603 = 400 rows of data
for i in range(60,len(test_data)):
x_test.append(test_data[i-60:i,0])
```

OUTPUT:

AMZN Stock History
Adj. Close
1200
1000
800
600
400
200
0
US $
1997
2001
2005
2009
2013
2017
Date

VIVA VOICE QUESTIONS

Q. What are the three stages to build the hypotheses or model in machinelearning?

Ans. Model building, Model testing, Applying the model.

Q. What is 'Training set' and 'Test set'?

Ans. In various areas of information science like machine learning, a set of data is used to discover the potentially predictive relationship known as 'Training Set'. Training set is an examples given to the learner, while Test set is used to test the accuracy of the hypotheses generated by the learner, and it is the set of example held back from the learner. Training setare distinct from Test set.

Q. What is the difference between artificial learning and machine learning?

Ans. Designing and developing algorithms according to the behaviours based on empiricaldata are known as Machine Learning. While artificial intelligence in addition to machine learning, it also covers other aspects like knowledge representation, natural language processing, planning, robotics etc.

Q. What is the general principle of an ensemble method and what is bagging andboosting in ensemble method?

Ans. The general principle of an ensemble method is to combine the predictions of several modelsbuilt with a given learning algorithm in order to improve robustness over a single model. Bagging is a method in ensemble for improving unstable estimation or classification schemes. While boosting method are used sequentially to reduce the bias of the combined model. Boosting and Bagging both can reduce errors by reducing the variance term.

Q. How do you choose an algorithm for a classification problem?

Ans. The answer depends on the degree of accuracy needed and the size of the training set. If you have a small training set, you can use a low variance/high bias classifier. If your trainingset is large, you will want to choose a high variance/low bias classifier.

EXPERIMENT-11

AIM - Case Study of RMS Titanic Database to predict survival on basis of decision tree, Logistic Regression , KNN or k-Nearest Neighbors, Support Vector Machines

The sinking of the **RMS Titanic** is one of the most infamous shipwrecks in history. On April 15,

1912, during her maiden voyage, the Titanic sank after colliding with an iceberg, killing 1502 out of 2224 passengers and crew. This sensational tragedy shocked the international community and led to better safety regulations for ships.One of the reasons that the shipwreck led to such loss of life was that there were not enough lifeboats for the passengers and crew. Although there was some element of luck involved in surviving the sinking, some groups of people were more likely to survive than others, such as women, children, and the upper-class.

	Survived	Pclass	Sex	Age	Fare	Embarked	Title	IsAlone	Age*Class
0	0	3	0	1	0	0	1	0	3
1	1	1	1	2	3	1	3	0	2
2	1	3	1	1	1	0	2	1	3
3	1	1	1	2	3	0	3	0	2
4	0	3	0	2	1	0	1	1	6
5	0	3	0	1	1	2	1	1	3
6	0	1	0	3	3	0	1	1	3
7	0	3	0	0	2	0	4	0	0
8	1	3	1	1	1	0	3	0	3
9	1	2	1	0	2	1	3	0	0

- **Survived** (Target Variable) - Binary categorical variable where 0 represents not survived and 1represents survived.
- **Pclass** - Categorical variable. It is passenger class.
- **Sex** - Binary Variable representing the gender the of passenger
- **Age** - Feature engineered variable. It is divided into 4 classes.
- **Fare** - Feature engineered variable. It is divided into 4 classes.
- **Embarked** - Categorical Variable. It tells the Port of embarkation.
- **Title** - New feature created from names. The title of names is classified into 4 different classes.
- **isAlone** - Binary Variable. It tells whether the passenger is travelling alone or not.
- **Age*Class** - Feature engineered variable.

Model, predict and solve

Now we are ready to train a model and predict the required solution. There are 60+ predictive modelling algorithms to choose from. We must understand the type of problem and solution requirement to narrow

down to a select few models which we can evaluate. Our problem is a classification and regression problem. We want to identify relationship between output (Survived or not) with other variables or features (Gender, Age, Port...). We are also performing a category of machine learning which is called supervised learning as we are training our model with a given dataset. With these two criteria - Supervised Learning plus Classification and Regression, we can narrow down our choice of models to a few. These include:

- Logistic Regression
- KNN or k-Nearest Neighbours
- Support Vector Machines
- Naive Bayes classifier
- Decision Tree

```
X_train = train_df.drop("Survived", axis=1)
Y_train = train_df["Survived"]
X_test  = test_df.drop("PassengerId", axis=1).copy()
X_train.shape, Y_train.shape, X_test.shape
```

```
((891, 8), (891,), (418, 8))
```

Size of the training and testing dataset

1. Logistic Regression is a useful model to run early in the workflow. Logistic regression measures the relationship between the categorical dependent variable (feature) and one or more independent variables (features) by estimating probabilities using a logistic function, which is the cumulative logistic distribution.
   Note the confidence score generated by the model based on our training dataset.

```
# Logistic Regression

logreg = LogisticRegression()
logreg.fit(X_train, Y_train)
Y_pred = logreg.predict(X_test)
acc_log = round(logreg.score(X_train, Y_train) * 100, 2)
acc_log
```

```
80.359999999999999
```

2. In pattern recognition, the k-Nearest Neighbours algorithm (or k-NN for short) is a non-parametric method used for classification and regression. A sample is classified by a majority vote of its neighbours, with the sample being assigned to the class most common among its k nearest neighbours (k is a positive integer, typically small). If k = 1, then the object is simply assigned to the class of that single nearest neighbour.
   KNN confidence score is better than Logistics Regression but worse than SVM.

```
knn = KNeighborsClassifier(n_neighbors = 3)
knn.fit(X_train, Y_train)
Y_pred = knn.predict(X_test)
acc_knn = round(knn.score(X_train, Y_train) * 100, 2)
acc_knn
```

```
84.739999999999995
```

3. Next we model using Support Vector Machines which are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis. Given a set of training samples, each marked as belonging to one or the other of **two categories**, an SVM training algorithm builds a model that assigns new test samples to one category or the other, making it a non-probabilistic binary linear classifier.

```
# Support Vector Machines

svc = SVC()
svc.fit(X_train, Y_train)
Y_pred = svc.predict(X_test)
acc_svc = round(svc.score(X_train, Y_train) * 100, 2)
acc_svc
```

```
83.840000000000003
```

Note that the model generates a confidence score which is higher than Logistics Regression model.

4. In machine learning, naive Bayes classifiers are a family of simple probabilistic classifiers based on applying Bayes' theorem with strong (naive) independence assumptions between the features. Naive Bayes classifiers are highly scalable, requiring a number of parameters linear in the number of variables (features) in a learning problem.
The model generated confidence score is the lowest among the models evaluated so far.

```
# Gaussian Naive Bayes

gaussian = GaussianNB()
gaussian.fit(X_train, Y_train)
Y_pred = gaussian.predict(X_test)
acc_gaussian = round(gaussian.score(X_train, Y_train) * 100, 2)
acc_gaussian
```

```
72.280000000000001
```

5. This model uses a decision tree as a predictive model which maps features (tree branches) to conclusions about the target value (tree leaves). Tree models where the target variable can take a finite set of values are called classification trees; in these tree structures, leaves represent class labels and branches represent conjunctions of features that lead to those class labels. Decision trees where the target variable can take continuous values (typically real numbers) are called regression trees. The model confidence score is the highest among models evaluated so far.

```
# Decision Tree

decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train, Y_train)
Y_pred = decision_tree.predict(X_test)
acc_decision_tree = round(decision_tree.score(X_train, Y_train) * 100, 2)
acc_decision_tree
```

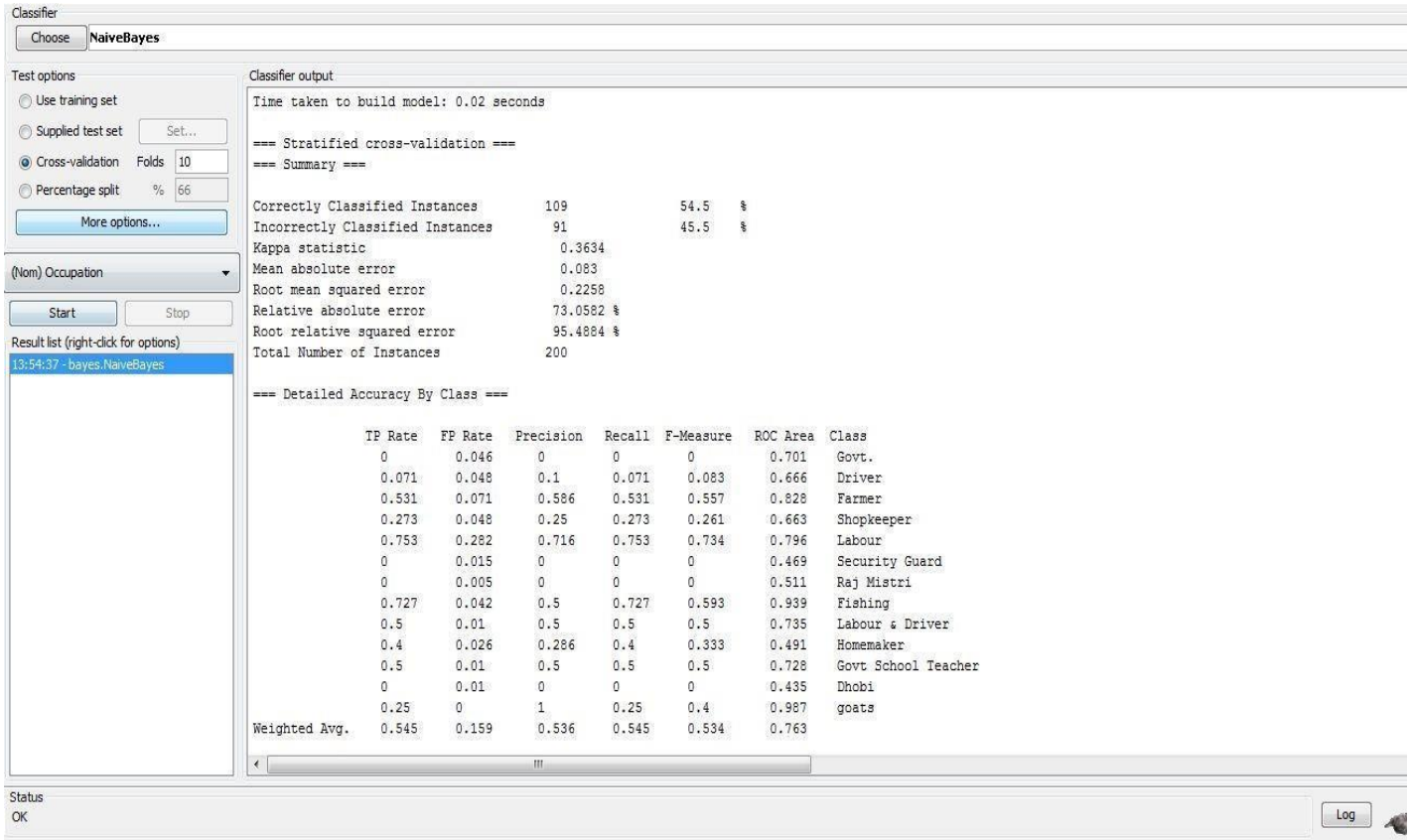
```
86.760000000000005
```

EXPERIMENT-12

AIM - Understanding of Indian education in Rural villages to predict whether girl child will be sent to school or not ?.

The data is focused on rural India. It primarily looks into the fact whether the villagers are willing to send the girl children to school or not and if they are not sending their daughters to school the reasons have also been mentioned. The district is Gwalior. Various details of the villagers such as village, gender, age, education, occupation, category, caste, religion, land etc have also been collected.

1 Naïve Bayes Classifier



The algorithm was run with 10-fold cross-validation: this means it was given an opportunity to make a prediction for each instance of the dataset (with different training folds) and the presented result is a summary of those predictions. Firstly, I noted the Classification Accuracy. The model achieved a result of 109/200 correct or 54.5%.

```
=== Confusion Matrix ===

  a  b  c  d  e  f  g  h  i  j  k  l  m   <-- classified as
  0  0  1  1  0  1  0  2  0  0  0  0  0 |  a = Govt.
  2  1  1  1  8  0  0  0  0  1  0  0  0 |  b = Driver
  2  0 17  2  9  0  0  2  0  0  0  0  0 |  c = Farmer
  0  0  4  3  2  0  1  0  0  1  0  0  0 |  d = Shopkeeper
  1  8  2  3 73  1  0  1  1  3  2  2  0 |  e = Labour
  3  0  0  0  0  0  0  1  0  0  0  0  0 |  f = Security Guard
  0  1  0  1  0  0  0  2  0  0  0  0  0 |  g = Raj Mistri
  1  0  0  0  1  1  0  8  0  0  0  0  0 |  h = Fishing
  0  0  2  0  0  0  0  0  2  0  0  0  0 |  i = Labour & Driver
  0  0  2  0  1  0  0  0  0  2  0  0  0 |  j = Homemaker
  0  0  0  0  1  0  0  0  1  0  2  0  0 |  k = Govt School Teacher
  0  0  0  1  4  0  0  0  0  0  0  0  0 |  l = Dhobi
  0  0  0  0  3  0  0  0  0  0  0  0  1 |  m = goats
```

The confusion matrix shows the precision of the algorithm showing that 1,1,1,2 Government officials were misclassified as Farmer, Shopkeeper, Security Guard and Fishermen respectively, 2,1,1,8,1 Drivers were misclassified as Government officials, Farmer, Shopkeeper, Labour, Homemaker, and so on. This table can help to explain the accuracy achieved by the algorithm.

Now when we have model, we need to load our test data we've created before. For this, select Supplied test set and click button Set. Click More Options, where in new window, choose PlainText from Output predictions. Then click left mouse button on recently created model on result list and select Re- evaluate model on current test set. After re-evaluation

Classifier output

```
   200   5:Labour   5:Labour          0      0.014  0.267  0.017 *0.529  0      0      0      0.029  0.091  0      0.054  0

=== Summary ===

Correctly Classified Instances         151               75.5    %
Incorrectly Classified Instances        49               24.5    %
Kappa statistic                          0.6634
Mean absolute error                      0.0554
Root mean squared error                  0.1649
Total Number of Instances              200

=== Detailed Accuracy By Class ===

               TP Rate   FP Rate   Precision   Recall  F-Measure   ROC Area  Class
                 0.8       0.021      0.5       0.8       0.615      0.994    Govt.
                 0.5       0.032      0.538     0.5       0.519      0.915    Driver
                 0.625     0.012      0.909     0.625     0.741      0.945    Farmer
                 0.636     0.026      0.583     0.636     0.609      0.929    Shopkeeper
                 0.825     0.175      0.816     0.825     0.821      0.901    Labour
                 0.5       0          1         0.5       0.667      0.999    Security Guard
                 1         0.005      0.8       1         0.889      1        Raj Mistri
                 1         0.037      0.611     1         0.759      0.999    Fishing
                 1         0          1         1         1          1        Labour & Driver
                 0.8       0.015      0.571     0.8       0.667      0.944    Homemaker
                 1         0.015      0.571     1         0.727      0.997    Govt School Teacher
                 0         0          0         0         0          0.981    Dhobi
                 1         0          1         1         1          1        goats
Weighted Avg.    0.755     0.094      0.759     0.755     0.746      0.931
```

Now the Classification Accuracy is 151/200 correct or 75.5%.

TP = true positives: number of examples predicted positive that are actually positive FP = false positives: number of examples predicted positive that are actually negative
TN = true negatives: number of examples predicted negative that are actually negative FN = false negatives: number of examples predicted negative that are actually positive
Recall is the TP rate (also referred to as sensitivity) what fraction of those that are actually positivewere predicted positive? : TP / actual positives Precision is TP / predicted Positive
what fraction of those predicted positive are actually positive? **precision** is also referred to as Positive predictive value (PPV); Other related measures used in classification include True Negative Rate and Accuracy: True Negative Rate is also called **Specificity**. (TN / actual negatives) 1-specificity is x-axis of ROC curve: this is the same as the FP rate (FP / actual negatives) **F- measure** A measure that combines precision and recall is the harmonic mean of precision and recall, the traditional F-measure or balanced F-score:

$$F = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

$$\text{Recall} = \frac{tp}{tp + fn}$$

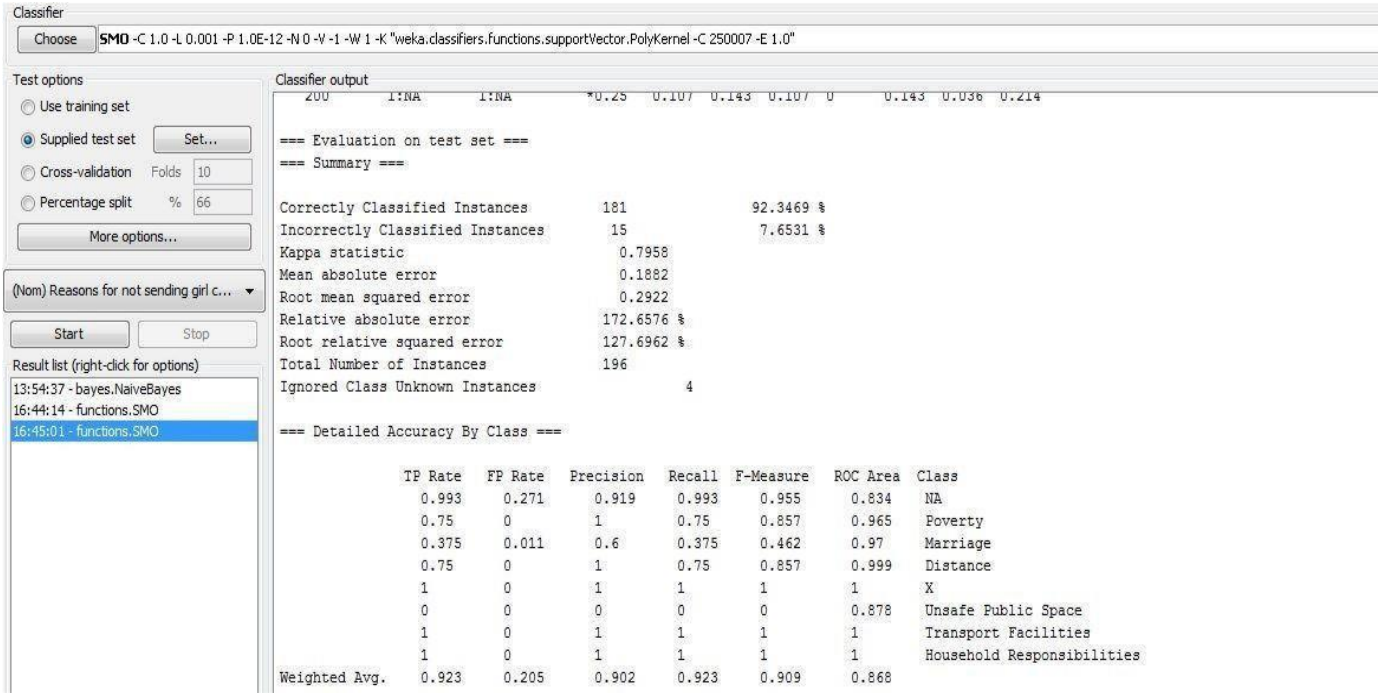
$$\text{Precision} = \frac{tp}{tp + fp}$$

$$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$$

Mean absolute error (MAE)
The MAE measures the average magnitude of the errors in a set of forecasts, without considering their direction. It measures accuracy for continuous variables. The equation is given in the library references. Expressed in words, the MAE is the average over the verification sample of the absolute values of the differences between forecast and the corresponding observation. The MAEis a linear score which means that all the individual differences are weighted equally in the average.

Root mean squared error (RMSE)
The RMSE is a quadratic scoring rule which measures the average magnitude of the error. The equation for the RMSE is given in both of the references. Expressing the formula in words, the difference between forecast and corresponding observed values are each squared and then averaged over the sample. Finally, the square root of the average is taken. Since the errors are squared before they are averaged, the RMSE gives a relatively high weight to large errors. This means the RMSE is most useful when large errors are particularly undesirable.

2 Support Vector Machine



The model achieved a result of 181/200 correct or 92.3469%.

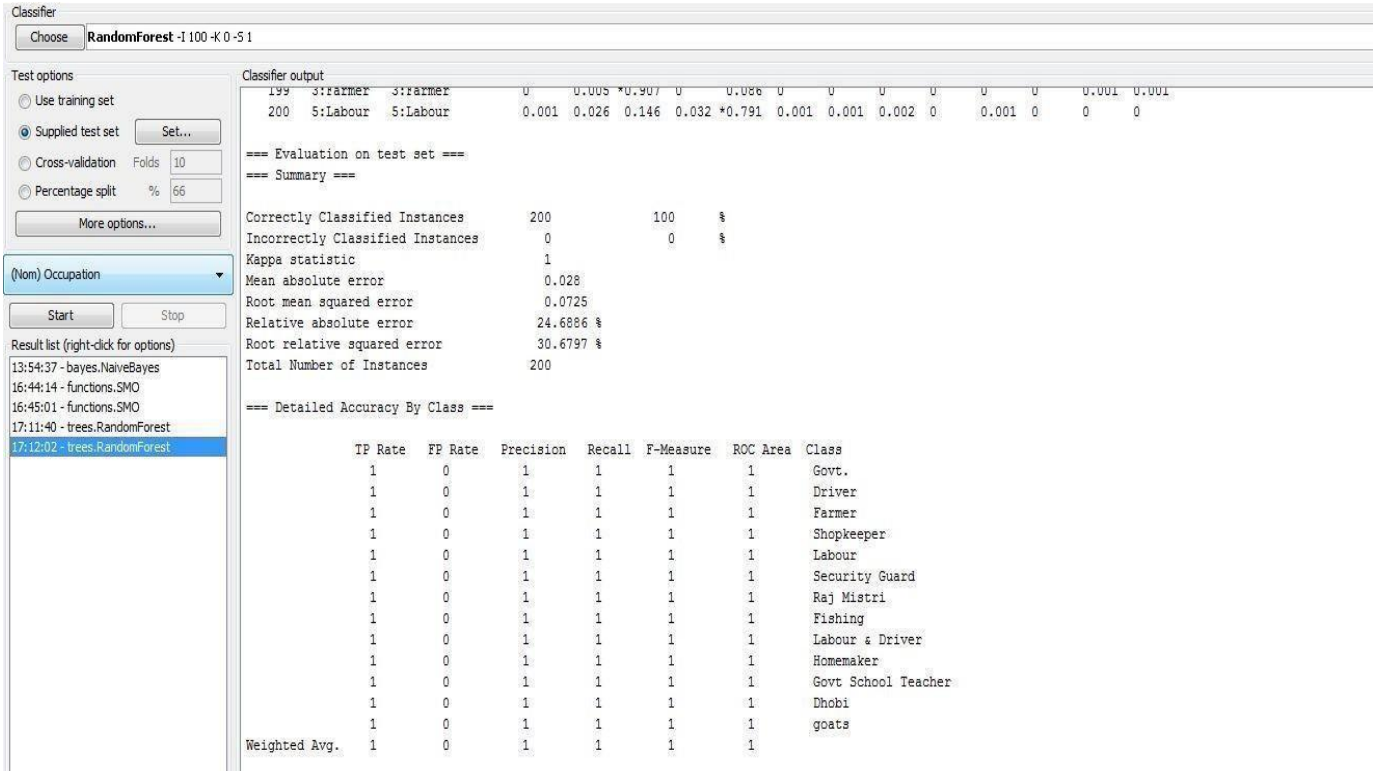
We have classified the dataset on the basis the reasons why the villagers are unwilling to send girl children to schools in Gwalior village. The different classes are NA, Poverty, Marriage, Distance, X, Unsafe Public Space, Transport Facilities, and Household Responsibilities.

The weighted average true positive rate is 0.923 that is nearly all the predicted positive values are actually positive. The weighted average false positive rate is 0.205 that is few of them are predicted as positive values but are actually negative. The precision in 0.902 that is the algorithmis nearly accurate.

```
=== Confusion Matrix ===

   a   b   c   d   e   f   g   h   <-- classified as
 147   0   1   0   0   0   0   0 |   a = NA
   4  12   0   0   0   0   0   0 |   b = Poverty
   5   0   3   0   0   0   0   0 |   c = Marriage
   0   0   1   3   0   0   0   0 |   d = Distance
   0   0   0   0   8   0   0   0 |   e = X
   4   0   0   0   0   0   0   0 |   f = Unsafe Public Space
   0   0   0   0   0   0   4   0 |   g = Transport Facilities
   0   0   0   0   0   0   0   4 |   h = Household Responsibilities
```

The confusion matrix shows that majority of the reasons were not available and out of the reasons which were available people did not send their daughters to school because of poverty and very few of them considered Distance as a major factor for not sending their girl children to school.

3 Random Forest

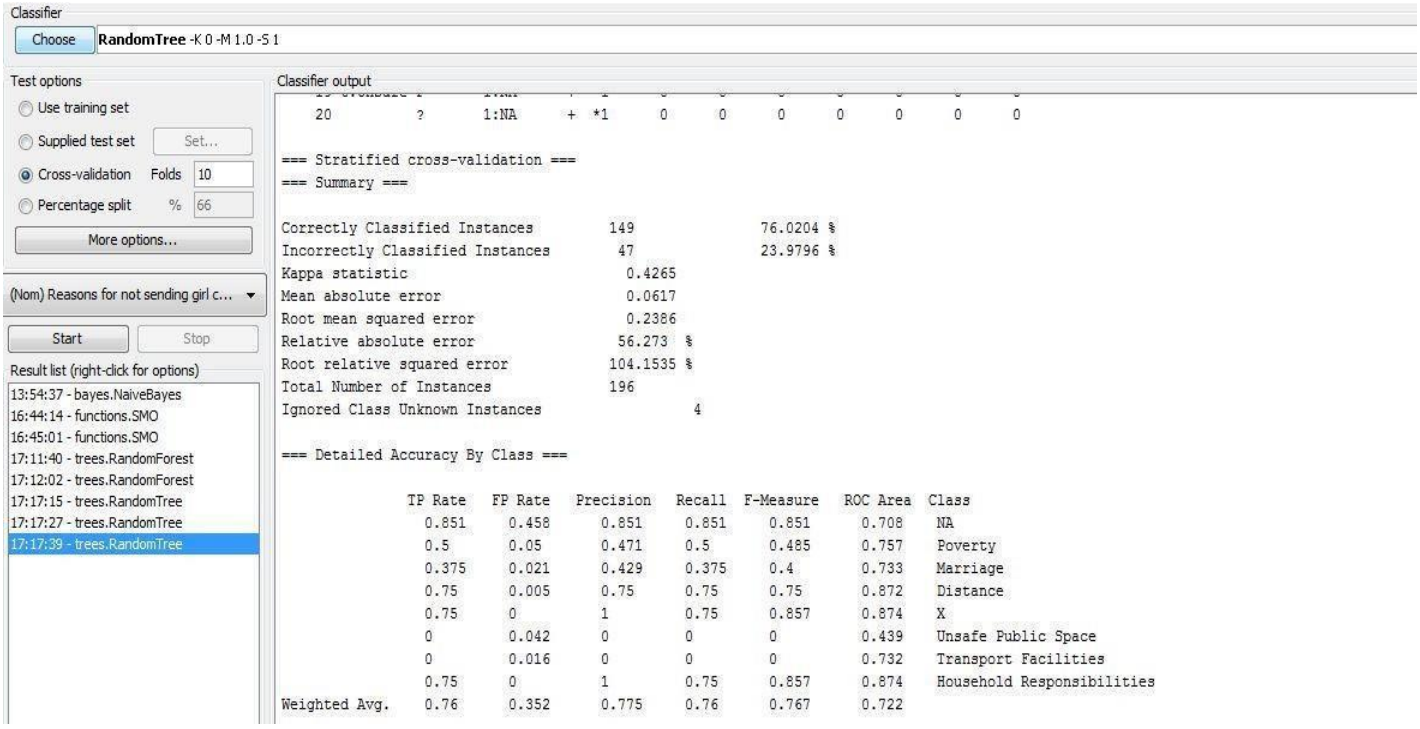


The accuracy of this algorithm is 100% that is 200/200 have been correctly classified

```
=== Confusion Matrix ===

  a  b  c  d  e  f  g  h  i  j  k  l  m   <-- classified as
  5  0  0  0  0  0  0  0  0  0  0  0  0 |  a = Govt.
  0 14  0  0  0  0  0  0  0  0  0  0  0 |  b = Driver
  0  0 32  0  0  0  0  0  0  0  0  0  0 |  c = Farmer
  0  0  0 11  0  0  0  0  0  0  0  0  0 |  d = Shopkeeper
  0  0  0  0 97  0  0  0  0  0  0  0  0 |  e = Labour
  0  0  0  0  0  4  0  0  0  0  0  0  0 |  f = Security Guard
  0  0  0  0  0  0  4  0  0  0  0  0  0 |  g = Raj Mistri
  0  0  0  0  0  0  0 11  0  0  0  0  0 |  h = Fishing
  0  0  0  0  0  0  0  0  4  0  0  0  0 |  i = Labour & Driver
  0  0  0  0  0  0  0  0  0  5  0  0  0 |  j = Homemaker
  0  0  0  0  0  0  0  0  0  0  4  0  0 |  k = Govt School Teacher
  0  0  0  0  0  0  0  0  0  0  0  5  0 |  l = Dhobi
  0  0  0  0  0  0  0  0  0  0  0  0  4 |  m = goats
```

There is no observation which has been misclassified. Maximum number of villagers are laborers

4 Random Tree



The classification accuracy is 76.0204% that is 149/200 have been classified correctly.
The false positive rate is 0.352 that is highest of all the four algorithms applied above. Here 35.2% of the values which should have been classified negatively have been assigned a positive value.

```
=== Confusion Matrix ===

   a   b   c   d   e   f   g   h   <-- classified as
 126   7   3   1   0   8   3   0 |   a = NA
   7   8   1   0   0   0   0   0 |   b = Poverty
   4   1   3   0   0   0   0   0 |   c = Marriage
   1   0   0   3   0   0   0   0 |   d = Distance
   2   0   0   0   6   0   0   0 |   e = X
   4   0   0   0   0   0   0   0 |   f = Unsafe Public Space
   3   1   0   0   0   0   0   0 |   g = Transport Facilities
   1   0   0   0   0   0   0   3 |   h = Household Responsibilities
```

22 NA , 8 Poverty , 5 Marriage, 1 Distance, 2 X, 4 Unsafe Public Space, 4 Transport Facilities and 1 Household Responsibilities class values have been misclassified.

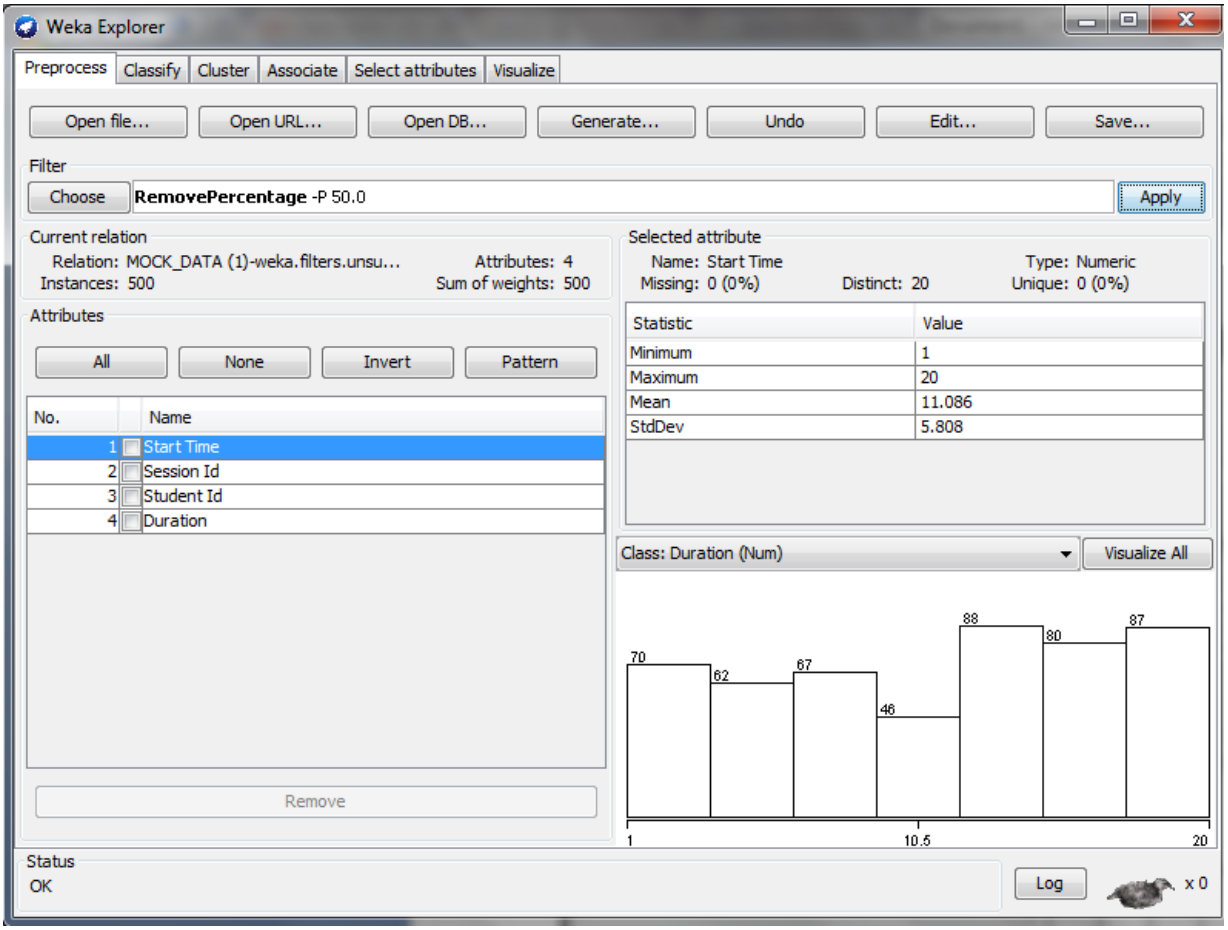
The best algorithm out of the above algorithms is Random Forest with 100% accuracy rate and theworst is Naïve Bayes algorithm with 75.5% accuracy rate.

EXPERIMENT-13

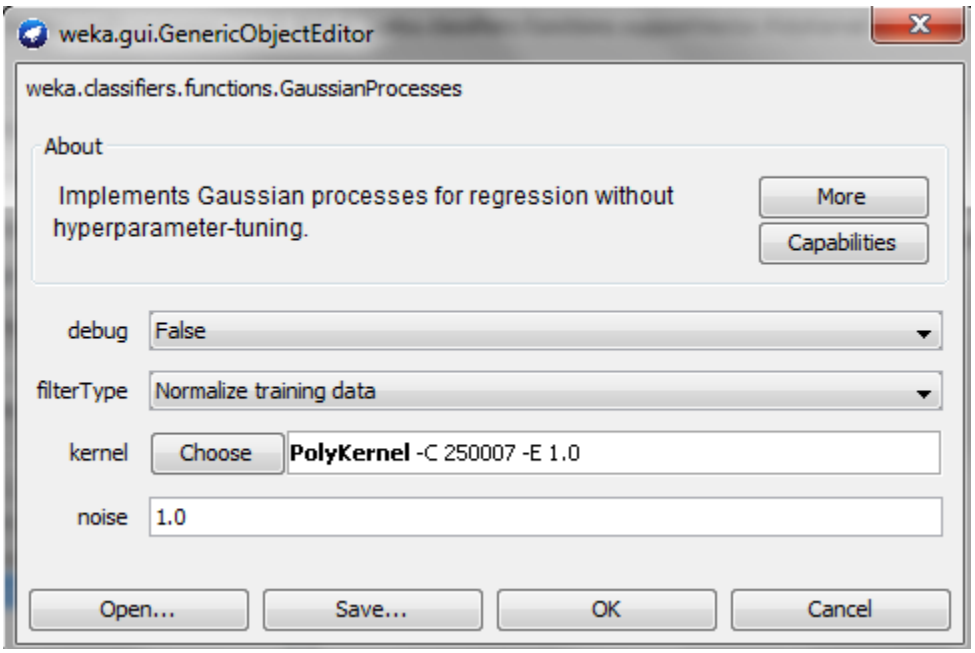
AIM - Understanding of dataset of contact patterns among students collected in NationalUniversity of Singapore.

This is dataset collected from contact patterns among students collected during the spring semester2006 in National University of Singapore

Using RemovePercentage filter, instances have been reduced to: 500
This data has been taken and saved as training data set and then used for further classification.



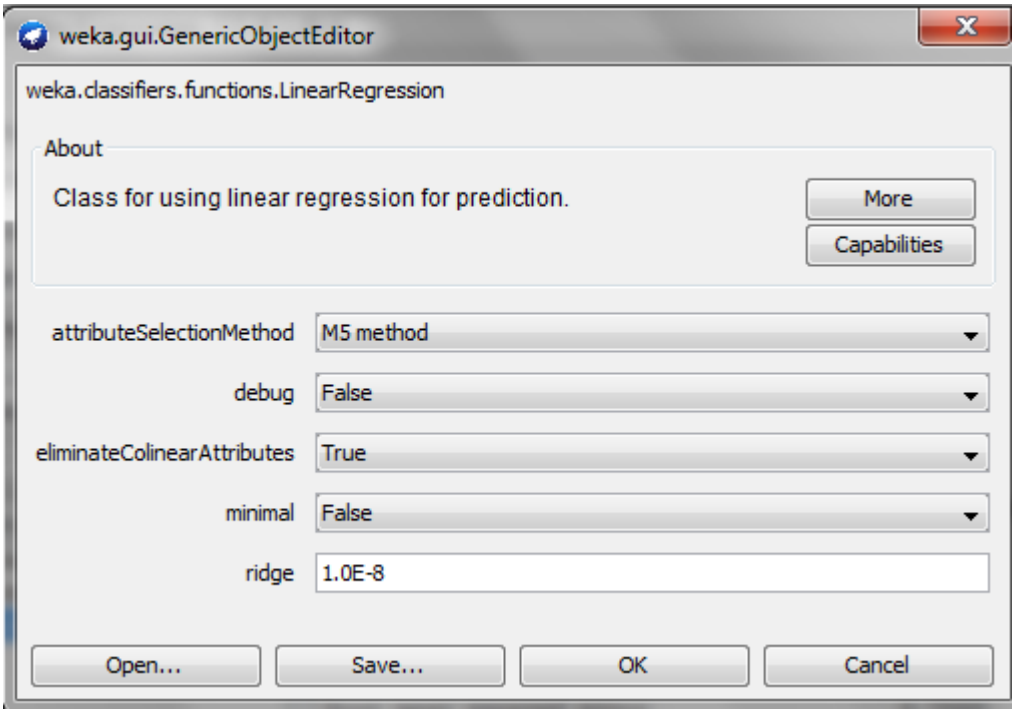
ALGORITHM-1 : Gaussian Processes




```
=== Run information ===

Scheme:         weka.classifiers.functions.SimpleLinearRegression
Relation:                                       MOCK_DATA       (1)-
weka.filters.unsupervised.instance.RemovePercentage-P50.0Instances:
500
Attributes:  4
Start TimeSession IdStudent IdDuration
Test mode: evaluate on training data

=== Classifier model (full training set) ===Linear regression on

Session Id

0.03 * Session Id + 10.38


Time taken to build model: 0 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds
=== Summary ===
```

```
orrelation coefficient                    0.0677
ean absolute error                        4.9869
oot mean squared error                    5.7893
elative absolute error                   99.7326 %
oot relative squared error               99.7708 %
otal Number of Instances               500
```

ALGORITHM 2: Linear Regression

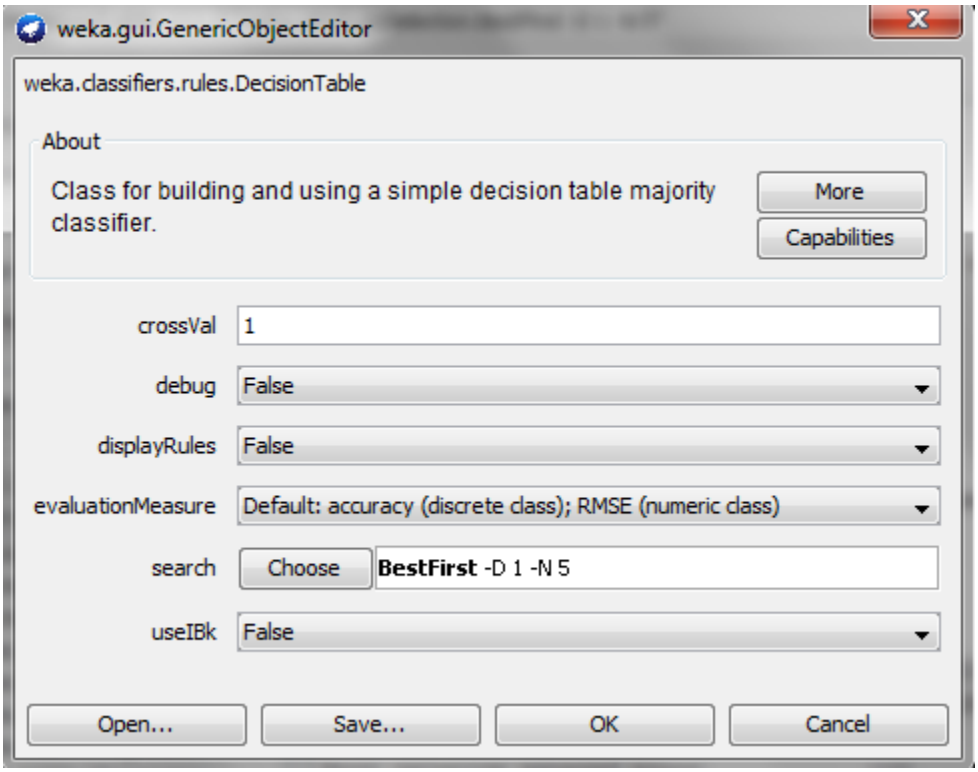


Linear Regression Model

```
Start Time =

 0.0274 * Session Id +10.3846

Time taken to build model: 0.01 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

== Summary ===

orrelation coefficient                    0.0677
ean absolute error                        4.9869
oot mean squared error                    5.7893
```

```
Relative absolute error                  99.7326 %
Root relative squared error              99.7708 %
Total Number of Instances               500
Algorithm 3:  Decision Table
```

Algorithm 3: DecisionTable



Merit of best subset found: 5.814
Evaluation (for feature selection): CV (leave one out)Feature set: 1

Time taken to build model: 0.02 seconds

```
=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correlation coefficient                   0
Mean absolute error                       5.0003
Root mean squared error               5.8026
Relative absolute error                   100      %Root relative
squared error                          100%
Total Number of Instances              500
```

CONCLUSION:

Three algorithms have been used to measure the best classifier. Depending on various attributes, performance of various algorithms can be measured via mean absolute error and correlation coefficient.

Depending on the results above, worst correlation has been found by DecisionTable and best correlation has been found by Decision Stump

```
=== Run information ===

Scheme:              weka.classifiers.rules.DecisionTable  -X  1  -S
"weka.attributeSelection.BestFirst -D 1 -N 5"
Relation:                                       MOCK_DATA       (1)-
weka.filters.unsupervised.instance.RemovePercentage-P50.0Instances:
500
Attributes:  4
Start TimeSession IdStudent IdDuration
Test mode: evaluate on training data

=== Classifier model (full training set) ===Decision Table:

Number of training instances: 500Number of Rules : 1
Non matches covered by Majority class.Best first.

Start set: no attributes Search direction: forward

Stale search after 5 node expansionsTotal number of subsets evaluated:

9
```